SISTEMI INFORMATIVI SU WEB · A.A. 2025/2026

FigureDB

La versione semplificata (figuredb_siw_sempl) spiegata classe per classe e collegata alla teoria del corso

Cosa trovi qui. Ogni package, ogni classe e ogni template del progetto `figuredb_siw_sempl`, con il *perché* di ogni scelta. Ogni scelta rimanda alle slide del corso (prof. Merialdo; per React, M. Napoleone). Il progetto ha la stessa struttura del progetto di riferimento del corso (SIWFestival): template "piatti", `@Autowired` sui campi, `data.sql` per i dati iniziali, widget React costruito con Vite. In fondo trovi i punti deboli, le differenze dalla specifica originale, una raccolta di domande probabili e una mappa "slide → codice".

Come leggere i riquadri

TEORIA collegamento esplicito a una slide del corso: il numero tra parentesi quadre indica la lezione e la slide, ad esempio *06 · sl. 35* = lezione 06 "Spring Repository", slide 35.

ATTENZIONE punto debole, incoerenza o differenza rispetto alla specifica.

ALL'ORALE cosa ti possono chiedere su quel punto e come rispondere.

Progetto di Damiano Nanni · documento generato il 29 ottobre 2026

Indice

1. Il quadro generale: architettura, layer e struttura delle cartelle
2. Configurazione: `pom.xml`, `application.properties`, `message.properties`
3. Il modello (package `model`): entità, mapping, validazione
4. Il Persistence Layer (package `repository`)
5. Il Service Layer (package `service`)
6. Il Controller Layer MVC (package `controller`)
7. Le viste Thymeleaf e il CSS
8. Sicurezza (package `security`)
9. API REST (package `rest` e `dto`)
10. Il frontend React (`frontend/react-figure`)
11. Dati iniziali (`data.sql`) e analisi delle strategie di fetch
12. Test automatici
13. Due casi d'uso seguiti dall'inizio alla fine
14. Punti deboli e differenze dalla specifica
15. Domande probabili all'orale
16. Mappa rapida: concetto delle slide → dove si trova nel codice

Lezioni citate: 01 Panoramica · 02/09 HTML e CSS · 03 Spring Boot (parti 1 e 2) · 04 JPA Object-Relational Mapping (parti 1 e 2) · 05 JPQL · 06 Spring Repository · 07 Transazioni e Service Layer · 08 Controller, Validazione, Gestione errori · 10 Autenticazione e autorizzazione · 11 REST · 12 React (parti 1, 2, 3).

1. Il quadro generale: architettura e layer

1.1 Che cosa fa il sistema

FigureDB è un sistema informativo su Web che cataloga **figure da collezione** (statuette), i **fumetti** e le **serie** (manga, anime, videogiochi…) da cui derivano, e le **aziende** che producono le figure. Ogni fumetto ha uno **scrittore**, uno o più **disegnatori** e un **editore**; ogni serie ha i suoi **personaggi**. Gli utenti registrati possono **recensire** le figure; gli amministratori gestiscono il catalogo e moderano le recensioni da un pannello dedicato.

Rispetto al progetto originale `figuredb_siw`, questa versione:

- **non ha l'entità `Gacha`** né alcun riferimento a essa (repository, service, pagine, immagini);
- **non ha il campo `edizioneLimitata`** sulle figure, né il relativo badge;
- ha un widget React di ricerca **senza filtri di prezzo minimo e massimo**;
- ha una nuova pagina admin **`/admin/recensioni`** con tutte le recensioni del sito;
- segue lo stile del progetto di riferimento del corso: niente fragment Criteria API, niente seeder Java, niente DTO record, niente sottocartelle nei template.

Dimensioni del progetto: 59 file Java (la classe `FiguredbApplication`, 11 entità e 2 enumerazioni nel modello, 11 repository, 11 service con 3 eccezioni di dominio, 14 classi nel package controller, 1 controller REST, 3 DTO, 2 classi di sicurezza), 33 template Thymeleaf, 1 foglio di stile, 1 widget React in TypeScript, 1 file `data.sql`.

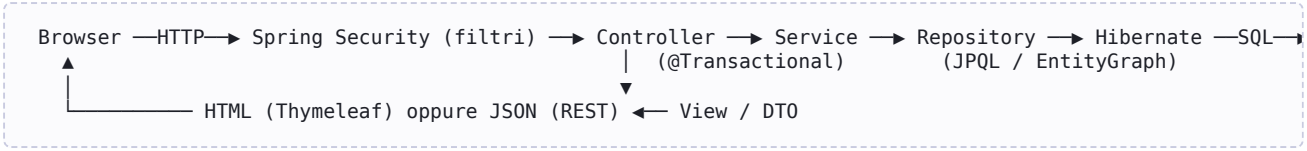
1.2 L'architettura a strati

> TEORIA
>
> 03 · sl. 3 **Anatomia di un SIW.** Il prof descrive l'applicazione a "quote" sempre più basse: architettura **client-server** (browser ↔ server), protocollo **HTTP + URI + cookie**, **HTML + CSS** come formato di risposta, architettura **3-tier** (presentazione, logica, persistenza) e, dentro il logic tier, il pattern **Model-View-Controller** con **CRUD** e **ORM**. 08 · sl. 3 riassume le responsabilità dei layer: *Model* (classi di dominio e mapping), *Persistence Layer* (repository), *Service Layer* (casi d'uso e transazioni), *Controller Layer* (richieste HTTP), *View Layer* (template HTML).

FigureDB rispetta questa divisione package per package:

Layer (slide 08 · sl. 3)	Package FigureDB	Contenuto
Model	`model`	11 `@Entity` + enum `Materiale`, `TipoSerie`
Persistence Layer	`repository`	interfacce `JpaRepository`, query derivate, `@Query` JPQL, `@EntityGraph`
Service Layer	`service`	casi d'uso, `@Transactional`, 3 eccezioni di dominio
Controller Layer	`controller` (MVC), `rest` (REST)	`@Controller`, `@RestController`, 3 `@ControllerAdvice`
View Layer	`templates/*.html`, `static/`, `frontend/`	Thymeleaf, CSS, React (Vite + TypeScript)
(trasversale)	`security`, `dto`	Spring Security; oggetti di trasferimento per REST e per la pagina prestazioni

La regola che regge tutto è il **flusso unidirezionale delle dipendenze**: il controller conosce il service, il service conosce i repository, i repository conoscono le entità. Nessun controller tocca direttamente un repository: tutti i campi `@Autowired` dei controller sono service (l'unica eccezione è `PasswordEncoder` in `AuthenticationController`, che però è un bean di sicurezza, non un repository).



1.3 La struttura delle cartelle

```
figuredb_siw_sempl/
├── pom.xml                          dipendenze Maven (Spring Boot 4.1, Java 17)
├── frontend/react-figure/           sorgenti del widget React (Vite + TypeScript + Axios)
│   ├── src/main.tsx, api.ts, types.ts, components/FigureSearchWidget.tsx
│   └── vite.config.ts               la build scrive in src/main/resources/static/js/react-figure/
└── src/main/
    ├── java/it/uniroma3/siw/figuredb/
    │   ├── FiguredbApplication.java
    │   ├── model/  repository/  service/  controller/  rest/  dto/  security/
    └── resources/
        ├── application.properties   message.properties   data.sql
        ├── templates/               33 pagine piatte: listFigure, showFigure, formFigure, ...
        │   ├── fragments/navbar.html
        │   └── error/404.html, 500.html, errore.html
        └── static/                  css/stile.css, images/, js/react-figure/assets/index.js
```

I template seguono la convenzione del progetto di riferimento: **`listX`** per l'elenco, **`showX`** per il dettaglio, **`formX`** per inserimento e modifica (lo stesso template). Così, dato un URL, si trova subito la pagina: `/fumetti/3` → `FumettoController.show` → `showFumetto.html`.

1.4 Il ciclo MVC in una richiesta

> TEORIA
>
> 03 · sl. 29 **Model-View-Controller.** (1) il controller mappa la richiesta HTTP in un'azione sul modello; (2) il modello esegue l'operazione sui dati; (3) il controller sceglie la vista; (4) la vista genera la risposta.

Esempio concreto: `GET /figure/7`

1. `FigureController.show(7, model)` riceve la richiesta (passo 1).
2. Chiama `figureService.findByIdWithAziendaESerie(7)`, `recensioneService.findByFigureId(7)` e `recensioneService.votoMedio(7)` (passo 2): ogni metodo del service apre una transazione read-only e il repository esegue la query (con `@EntityGraph` per caricare azienda e serie insieme alla figure).
3. Inserisce i risultati nel `Model` con `addAttribute("figure", …)`, `addAttribute("recensioni", …)`, `addAttribute("votoMedio", …)` e restituisce la stringa `"showFigure"` (passo 3).
4. Thymeleaf elabora `templates/showFigure.html` e produce l'HTML (passo 4).

Se la figure 7 non esiste, il service restituisce un `Optional` vuoto, il controller lancia `RisorsaNonTrovataException` e `GlobalExceptionHandler` mostra `error/404.html` con codice 404 (§6.4).

1.5 La classe `FiguredbApplication`

```java
@SpringBootApplication
public class FiguredbApplication {
    public static void main(String[] args) {
        SpringApplication.run(FiguredbApplication.class, args);
    }
}
```

TEORIA

03 · sl. 27 "Un progetto Spring Boot ha una classe Application, che rappresenta l'intera applicazione. Il metodo main lancia l'application server e carica tutti i componenti." Non serve installare Tomcat a parte: è incluso nel jar.

`@SpringBootApplication` riunisce tre annotazioni: `@Configuration`, `@EnableAutoConfiguration` (Spring configura da solo DataSource, JPA, Thymeleaf... in base alle dipendenze presenti) e `@ComponentScan`. Quest'ultima cerca `@Controller`, `@Service`, `@Component` e `@Configuration` **nel package `it.uniroma3.siw.figuredb` e nei sottopackage**: per questo tutte le classi stanno sotto quel package.

2. Configurazione

2.1 `pom.xml` : le dipendenze

> **TEORIA**
>
> 03 · sl. 19 Le librerie base sono Spring Web, PostgreSQL Driver, Thymeleaf e Spring Data JPA. 08 · sl. 17 aggiunge `spring-boot-starter-validation` ; 10 · sl. 5 aggiunge `spring-boot-starter-security` . 00 · sl. 21 Maven è il sistema di gestione del progetto.

Dipendenza	A cosa serve	Lezione
`spring-boot-starter-webmvc`	controller, Tomcat embedded, Jackson (JSON)	03, 11
`spring-boot-starter-data-jpa`	Hibernate + Spring Data (repository)	04–06
`spring-boot-starter-thymeleaf`	template engine delle viste	03, 08
`spring-boot-starter-validation`	Bean Validation (`@NotBlank` , `@Valid` ...)	08
`spring-boot-starter-security`	autenticazione e autorizzazione	10
`thymeleaf-extras-springsecurity6`	dialetto `sec:` nei template (`sec:authorize` , `sec:authentication`)	10
`postgresql` (runtime)	driver JDBC del DBMS	03
`spring-boot-devtools` (runtime)	riavvio automatico in sviluppo	—
`*-test` , `spring-security-test`	JUnit 5, Spring Test	—

Il `parent` è `spring-boot-starter-parent` 4.1.0: fissa le versioni compatibili di tutte le librerie, per questo le dipendenze non riportano `<version>` . `<java.version>17</java.version>` corrisponde al requisito "Java 17 o superiore" 03 · sl. 4 . Non c'è H2: il progetto gira solo su PostgreSQL, come quello di riferimento.

2.2 `application.properties`

> **TEORIA**
>
> 03 · sl. 23–25 I parametri di configurazione stanno in `application.properties` come coppie `nome=valore` : datasource, porta, Thymeleaf. 03 · sl. 77 `ddl-auto=create` ricrea il database a ogni avvio, comodo in sviluppo; in esercizio si usa `none` .

```
spring.datasource.url=jdbc:postgresql://localhost:5432/figuredb_sempl
spring.datasource.username=${DB_USERNAME:postgres}
spring.datasource.password=${DB_PASSWORD:postgres}
spring.jpa.hibernate.ddl-auto=update
spring.jpa.open-in-view=true
spring.jpa.defer-datasource-initialization=true
spring.sql.init.mode=always
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.generate_statistics=true
server.port=8081
spring.thymeleaf.cache=false
spring.messages.basename=message
```

Le scelte, una per una:

- **Database `figuredb_sempl` sulla porta 8081.** Nome del DB e porta sono diversi da quelli del progetto originale (`figuredb`, 8080), così le due versioni possono girare insieme senza pestarsi i piedi. Il database va creato vuoto (per esempio con pgAdmin) prima del primo avvio: le tabelle le crea Hibernate.
- **Credenziali da variabili d'ambiente.** `${DB_PASSWORD:postgres}` significa "usa la variabile `DB_PASSWORD`; se non esiste usa `postgres`". La password vera non finisce su Git.
- **`ddl-auto=update`**: a ogni avvio Hibernate allinea lo schema alle entità **senza cancellare le tabelle**, quindi recensioni e utenti registrati sopravvivono al riavvio.
- **`defer-datasource-initialization=true` + `sql.init.mode=always`**: Spring Boot esegue `data.sql` a **ogni** avvio, ma solo **dopo** che Hibernate ha creato le tabelle. Senza la prima proprietà lo script girerebbe prima di Hibernate e fallirebbe, perché le tabelle non esistono ancora. Gli INSERT dello script sono idempotenti (§11.1), quindi rieseguirli non duplica nulla.
- **`open-in-view=true`**: è la scelta più importante da saper spiegare. Con l'*Open Session in View* la sessione Hibernate resta aperta per tutta la richiesta HTTP, **fino alla fine del rendering del template**, anche dopo che la transazione del service si è chiusa. I template possono quindi navigare le associazioni LAZY: `${s.figure.size()}`, `${azienda.figure.size()}`, `${x.fumetti.size()}` funzionano ovunque; la prima volta che si accede alla collezione Hibernate esegue la query. Con `false` questi accessi lancerebbero una `LazyInitializationException` (errore 500), perché la sessione sarebbe già chiusa. Il costo sono le query "nascoste" durante il rendering: vedi il riquadro qui sotto e il §14.
- **`show-sql`, `format_sql`, `logging.level.org.hibernate.SQL=debug`**: ogni istruzione SQL compare nella console, formattata. È utile all'orale per far vedere le query generate da Hibernate.
- **`generate_statistics=true`**: Hibernate conta le query eseguite; il contatore serve alla pagina `/admin/performance` (§11.2).
- **`spring.thymeleaf.cache=false`** 03 · sl. 24: i template vengono riletti a ogni richiesta, quindi in sviluppo si vedono le modifiche senza riavviare.
- **`spring.messages.basename=message`**: dice a Spring dove trovare i messaggi di errore personalizzati, cioè `message.properties` (§2.3).

> ALL'ORALE
>
> **"Perché `open-in-view=true`? Non è considerato una cattiva pratica?"** È una scelta consapevole: con `true` tutti i conteggi nei template si fanno con `.size()` sulle collezioni LAZY, senza `LazyInitializationException`. I due rischi noti sono le query "nascoste" durante il rendering (problema N+1 06 · sl. 33–34) e la connessione al DB occupata per tutta la richiesta. In questo progetto la pagina `/serie` fa 1 query per le 17 serie + 17 per le figure + 17 per i fumetti. Si contiene con `@BatchSize(size = 25)` sulle collezioni (le 17 collezioni dello stesso tipo arrivano con una sola query `… where serie_id in (…)`) oppure, in un sistema ad alto carico, mettendo `open-in-view=false` e facendo i conteggi con una query aggregata (`select f.serie.id, count(f) from Figure f group by f.serie.id`, 05 · sl. 16–19). **"Che cosa succede se lo metti a `false`?"** Ogni `.size()` su una collezione LAZY nei template dà `LazyInitializationException` (500), e bisognerebbe caricare tutto nel service con `JOIN FETCH` o `@EntityGraph`.

2.3 `message.properties` : i messaggi di validazione

TEORIA

08 · sl. 26–27 **Personalizzazione dei messaggi di errore.** I messaggi stanno in un file di configurazione indicato in `application.properties` (`spring.messages.basename`). Ogni chiave è un codice di errore: `NotBlank` vale per tutti i campi, `NotBlank.movie` per i campi dell'entità `movie` , `NotBlank.movie.title` per il solo campo `title` . "In caso di conflitti viene usata sempre la chiave più specifica." `typeMismatch` copre gli errori di conversione (una lettera in un campo numerico).

```
NotBlank=Campo obbligatorio
NotNull=Campo obbligatorio
Positive=Deve essere un numero maggiore di zero
typeMismatch.java.lang.Float=Deve essere un numero
NotBlank.figureForm.nome=Il nome della figure e' obbligatorio
Pattern.fumettoForm.isbn=ISBN non valido: 10 o 13 cifre (l'ultima puo' essere X)
NotBlank.credentials.user.email=L'email e' obbligatoria
```

La parte centrale della chiave è il **nome dell'attributo del model**, non il nome della classe: la form delle figure lavora sull'attributo `figureForm` , quindi la chiave è `NotBlank.figureForm.nome` . Per i campi annidati si usa il percorso: la form di registrazione lavora su `credentials` che contiene `user` , quindi `NotBlank.credentials.user.email` . Le entità non hanno l'attributo `message` nelle annotazioni: il testo viene sempre da questo file.

Il commento in testa al file spiega un dettaglio tecnico: Spring usa `java.text.MessageFormat` , che accetta solo segnaposto numerici (`{0}`), non `{min}` / `{max}` di Bean Validation. Per questo i limiti sono scritti direttamente nel testo ("almeno 3 caratteri").

3. Il modello (package `model`)

3.1 Le regole comuni a tutte le entità

Le 11 entità (`Serie`, `Personaggio`, `Figure`, `Azienda`, `Fumetto`, `Scrittore`, `Disegnatore`, `Editore`, `Recensione`, `User`, `Credentials`) seguono lo stesso schema. Conviene capirlo una volta sola.

```java
@Entity
public class Azienda {
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;

    @NotBlank
    @Column(unique = true)
    private String nome;

    @Size(max = 2000)
    @Column(length = 2000)
    private String descrizione;
    ...
    public Azienda() { }            // costruttore no-arg obbligatorio

    @Override
    public int hashCode() { return Objects.hash(nome); }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || getClass() != obj.getClass()) return false;
        Azienda other = (Azienda) obj;
        return Objects.equals(nome, other.nome);
    }
}
```

TEORIA

04 · sl. 10–14 **Entity e ORM.** Si chiama *entità* una classe i cui oggetti vanno memorizzati in modo persistente. Ogni classe corrisponde a una tabella, ogni attributo di tipo semplice a una colonna, ogni oggetto a una riga. Vale la regola della **configuration by exception** 04 · sl. 13: si annota solo ciò che si discosta dal default. 04 · sl. 22 **Restrizioni:** costruttore pubblico senza argomenti, classe non `final`, getter e setter. 04 · sl. 23 `@Id` indica la chiave primaria; `@GeneratedValue(strategy = AUTO)` fa generare il valore al primo salvataggio; `@Column(nullable=false, unique=true, length=…)` serve **solo** quando si vuole cambiare il default.

Configuration by exception, applicata davvero. Molti campi non hanno nessuna `@Column`: `prezzo`, `altezza`, `anno`, `genere` diventano colonne con il nome e il tipo di default. `@Column` compare solo per `unique = true` (nomi, ISBN, email, username) e per `length = 2000` sulle descrizioni (il default sarebbe `varchar(255)`).

`equals` / `hashCode` sull'identificatore naturale. Il progetto confronta le entità sul campo che le identifica nel mondo reale: `nome` per serie e aziende, `isbn` per i fumetti, `email` per gli utenti, `username` per le credenziali, `nome + dataUscita` per le figure, `nome + cognome + dataNascita` per scrittori e disegnatori.

> **TEORIA**
>
> 04 · sl. 24 "Su quali attributi verifichiamo l'identità? L'id assume un valore solo dopo che l'oggetto viene persistito... ma nel database dobbiamo imporre unicità sull'**identificatore naturale**." Nell'esempio del prof (`Product`) l'identificatore naturale è `code` , annotato `unique` . 04 · sl. 20 mostra anche la variante sull' `id` .

Il vantaggio dell'identificatore naturale è che `equals` funziona anche su oggetti **non ancora salvati** (id nullo), per esempio quando la form di un fumetto controlla `fumettoForm.disegnatori.contains(d)` per preselezionare i disegnatori. `Recensione` è l'unica eccezione e confronta l' `id` : non ha un identificatore naturale semplice (la coppia figure-autore è garantita dal vincolo di unicità sulla tabella).

Due famiglie di annotazioni sullo stesso campo. Va distinta bene, perché è una domanda classica:

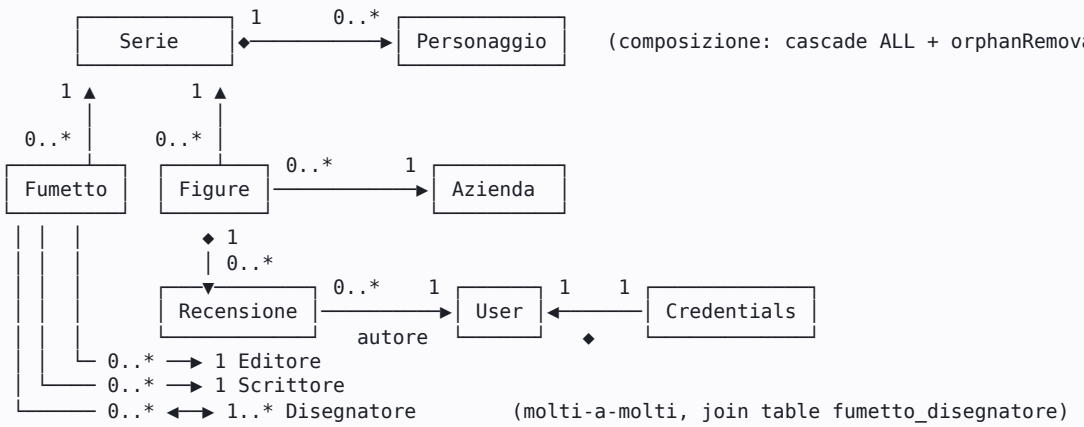
Annotazione	Package	Chi la controlla	Quando
`@Column(nullable = false, unique = true)`	`jakarta.persistence`	il **DBMS** (vincoli `NOT NULL` , `UNIQUE` nello schema)	all'INSERT/UPDATE
`@NotBlank` , `@NotNull` , `@Size` , `@Positive` …	`jakarta.validation.constraints`	**Bean Validation**, nel controller con `@Valid`	al binding della form

> **TEORIA**
>
> 08 · sl. 16, 18 "I vincoli sui dati di input sono proprietà delle entità... è possibile specificare in maniera dichiarativa ulteriori vincoli di validità direttamente sugli attributi". 08 · sl. 10 La validazione lato server è sempre necessaria, anche se c'è quella HTML5 lato client.

`@DateTimeFormat(iso = DateTimeFormat.ISO.DATE)` su `LocalDate` dice a Spring come convertire la stringa `2026-03-14` che arriva da `<input type="date">` in un oggetto `LocalDate` durante il binding [08 · sl. 6: "i parametri HTTP sono sempre stringhe... convertiti automaticamente nel tipo di destinazione"]. `LocalDate` viene mappato su una colonna `DATE` 04 · sl. 16–17 .

`@JsonIgnore` sulle collezioni. Tutte le liste `@OneToMany` / `@ManyToMany` lato inverso sono annotate `@JsonIgnore` : se un'entità venisse serializzata in JSON, Jackson non entrerebbe nelle collezioni (evitando cicli infiniti e caricamenti LAZY). È la soluzione proposta in 11 · sl. 23–28 . Le API REST, comunque, restituiscono DTO e non entità (§9).

3.2 Il diagramma delle classi



3.3 Tabella riassuntiva delle associazioni

Questa tabella è da conoscere a memoria per l'orale: per ogni associazione dice cardinalità, chi è il **lato proprietario** (owning side, quello con la foreign key), fetch e cascade.

Associazione	Annotazione (lato proprietario)	Lato inverso	Fetch	Cascade
Figure → Azienda	`@ManyToOne` + `@JoinColumn(name="azienda_id")` in `Figure`	`Azienda.figure` `mappedBy="azienda"`	LAZY	–
Figure → Serie	`@ManyToOne` in `Figure`	`Serie.figure`	LAZY	–
Figure ◆→ Recensione	`@ManyToOne` in `Recensione.figure`	`Figure.recensioni`	LAZY	REMOVE + orphanRemoval
Recensione → User (autore)	`@ManyToOne` in `Recensione`	(unidirezionale)	LAZY	–
Serie ◆→ Personaggio	`@ManyToOne` in `Personaggio`	`Serie.personaggi`	LAZY	ALL + orphanRemoval
Fumetto → Serie / Scrittore / Editore	`@ManyToOne` in `Fumetto`	`Serie.fumetti`, `Scrittore.fumetti`, `Editore.fumetti`	LAZY	–
Fumetto ↔ Disegnatore	`@ManyToMany` + `@JoinTable` in `Fumetto`	`Disegnatore.fumetti` `mappedBy="disegnatori"`	LAZY	–
Credentials ◆→ User	`@OneToOne` in `Credentials`	(unidirezionale)	EAGER (default)	ALL

> TEORIA
>
> 04 · sl. 69–72 **Owning side e `mappedBy`.** In un'associazione bidirezionale JPA distingue *parte proprietaria* (la tabella con la FK; in una one-to-many è il lato *many*) e *parte inversa*. La reciprocità si esprime con `@OneToMany(mappedBy = "…")` sul lato inverso. 04 · sl. 74–76 Senza `mappedBy`, JPA crede che le due collezioni siano **due associazioni diverse** e crea una tabella di join in più. 06 · sl. 23 **Default di fetch:** `@OneToOne` e `@ManyToOne` EAGER, `@OneToMany` e `@ManyToMany` LAZY. 06 · sl. 24 "In generale conviene usare i default… eventuali eccezioni devono essere giustificate dai casi d'uso." 06 · sl. 57 "Impostare cascade per PERSIST, MERGE solo su associazioni che esprimono una **composizione**."

Perché tutti i `@ManyToOne` sono LAZY, contro il default? Il default EAGER sembra innocuo (si tratta di un solo oggetto), ma la slide 06 · sl. 33–34 mostra che su un elenco di N oggetti Hibernate lo realizza con **1 + N query**. La regola del progetto è: tutto LAZY, e ogni query carica **esplicitamente** ciò che serve alla pagina, con `@EntityGraph` o `JOIN FETCH` (§4). Così ogni scelta di caricamento è visibile nel repository e motivata dal caso d'uso, come chiede 06 · sl. 24. Il commento su `Figure.azienda` lo dice: "LAZY di proposito: è l'associazione su cui si osserva il problema N+1 nella pagina di analisi delle prestazioni" (§11.2).

`@JoinColumn(name = "azienda_id")` fissa il nome della colonna FK. È uguale al default (`<attributo>_<pk>` 04 · sl. 58), quindi tecnicamente superfluo: rende però esplicito il nome usato in `@UniqueConstraint` e in `data.sql`.

3.4 Le entità una per una

`Serie`

L'opera di origine (manga, anime, videogioco…). È il **centro** del modello: ha tre collezioni inverse.

```java
@NotNull
@Enumerated(EnumType.STRING)
private TipoSerie tipo;

@OneToMany(mappedBy = "serie", cascade = CascadeType.ALL, orphanRemoval = true, fetch =
FetchType.LAZY)
@JsonIgnore
private List<Personaggio> personaggi = new ArrayList<>();

@OneToMany(mappedBy = "serie", fetch = FetchType.LAZY)
@JsonIgnore
private List<Fumetto> fumetti = new ArrayList<>();

@OneToMany(mappedBy = "serie", fetch = FetchType.LAZY)
@JsonIgnore
private List<Figure> figure = new ArrayList<>();
```

- `@Enumerated(EnumType.STRING)`: salva il nome della costante (`"MANGA"`) e non la posizione (`0`). Con `ORDINAL`, aggiungere un valore in mezzo all'enum sposterebbe il significato di tutte le righe già salvate. È la stessa scelta della slide `07 · sl. 4` (`OrderStatus`).
- `personaggi` con `cascade = ALL` + `orphanRemoval`: un personaggio non ha senso fuori dalla sua serie, quindi è una **composizione**. Salvare la serie salva i personaggi; cancellare la serie li cancella; togliere un personaggio dalla lista lo cancella dal DB. Corrisponde a `06 · sl. 49–57`.
- `fumetti` e `figure` **senza cascade**: un fumetto o una figure non vanno cancellati insieme alla serie. Il service impedisce di cancellare una serie che ha ancora fumetti o figure collegati (§5.5).
- **Le collezioni inizializzate con** `new ArrayList<>()`: un oggetto nuovo ha liste vuote e non `null`, quindi `serie.getFigure().size()` vale 0 senza `NullPointerException`.

`Personaggio`

```java
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "serie_id")
private Serie serie;
```

È il lato *many*, quindi è **proprietario**: la tabella `personaggio` ha la colonna `serie_id`. Il personaggio viene salvato da `SerieService.aggiungiPersonaggio`, che imposta `personaggio.setSerie(serie)` e chiama `personaggioRepository.save`: basta il lato proprietario, perché JPA scrive la FK leggendo solo quello.

`Figure`: l'entità centrale

```
@Entity
@Table(uniqueConstraints = @UniqueConstraint(name = "uk_figure_nome_azienda_data",
        columnNames = { "nome", "azienda_id", "data_uscita" }))
public class Figure {
    @NotBlank private String nome;
    @NotNull @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) private LocalDate dataUscita;
    @NotNull @Positive private Float altezza;
    @NotNull @Enumerated(EnumType.STRING) private Materiale materiale;
    private String immagine;
    @NotNull @Positive private Float prezzo;
    @Size(max = 2000) @Column(length = 2000) private String descrizione;

    @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "azienda_id") private Azienda
azienda;
    @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "serie_id")   private Serie serie;

    @OneToMany(mappedBy = "figure", cascade = CascadeType.REMOVE, orphanRemoval = true, fetch
= FetchType.LAZY)
    @JsonIgnore
    private List<Recensione> recensioni = new ArrayList<>();
}
```

- `@UniqueConstraint` **su tre colonne**: è il vincolo di consistenza della specifica ("la stessa azienda non può avere due figure con lo stesso nome e la stessa data di uscita"). Applica la sintassi di `04 · sl. 26, 52–54` per i vincoli di unicità su più attributi. I nomi delle colonne sono quelli **fisici**: Spring Boot traduce `dataUscita` in `data_uscita` (snake_case). Il vincolo è anche controllato nel service (§5.5): quello del DB è l'"ultima difesa".
- `immagine` è una stringa con il percorso (`/images/figure/<cartella>/<file>`), non il file: le immagini stanno in `static/images` e la pagina le mostra con `th:src="@{${f.immagine}}"`.
- `recensioni` **in composizione**: `CascadeType.REMOVE` + `orphanRemoval` fanno sì che cancellare una figure cancelli le sue recensioni. `FigureService.delete` si limita a `figureRepository.deleteById(id)`. Non c'è PERSIST: le recensioni si salvano una per una con `recensioneRepository.save`.
- **Rispetto all'originale** non c'è più `edizioneLimitata` né il `@NamedEntityGraph`: i grafi di caricamento sono dichiarati direttamente sui metodi del repository con `@EntityGraph(attributePaths = …)`.

`Azienda`, `Editore`

Entità semplici, con un nome unico, una `regione` obbligatoria, `descrizione` lunga (`@Column(length = 2000)` più `@Size(max = 2000)`, coerenti tra loro) e `logo` (un percorso, non il file). Hanno solo una collezione inversa LAZY senza cascade: `Azienda.figure`, `Editore.fumetti`.

`Fumetto`

```java
@NotBlank
@Pattern(regexp = "^(97(8|9))?\\d{9}(\\d|X)$")
@Column(unique = true)
private String isbn;

private boolean variant;

@ManyToMany(fetch = FetchType.LAZY)
@JoinTable(name = "fumetto_disegnatore",
        joinColumns = @JoinColumn(name = "fumetto_id"),
        inverseJoinColumns = @JoinColumn(name = "disegnatore_id"))
private List<Disegnatore> disegnatori = new ArrayList<>();
```

- **ISBN**: `unique = true` sullo schema e `@Pattern` per il formato. La regex accetta ISBN-10 (9 cifre + cifra o `X`) e ISBN-13 (prefisso `978` / `979`). `@PositiveOrZero` sul prezzo ammette un volume gratuito; `@Min(1900) @Max(2100)` sull'anno.
- `variant` è un `boolean` primitivo: una checkbox non selezionata non invia nulla e il campo resta `false`, senza problemi di `null`.
- **La molti-a-molti**: nel mondo relazionale si realizza con una tabella di join contenente due FK 04 · sl. 40, 83–86. `Fumetto` è il **lato proprietario** (ha `@JoinTable`), `Disegnatore` è l'inverso con `mappedBy = "disegnatori"`. `@JoinTable` rinomina tabella e colonne, che di default si chiamerebbero `fumetto_disegnatori` e `disegnatori_id`.
- **Senza cascade**: i disegnatori esistono indipendentemente dai fumetti.

> TEORIA
>
> 04 · sl. 83 "Una ManyToMany nel db è perfettamente simmetrica. Ma anche in questo caso è necessario specificare la reciprocità: se non lo facciamo JPA interpreta le due collezioni come due associazioni indipendenti." Togliendo `mappedBy` da `Disegnatore` si otterrebbero **due** tabelle di join (esercizio 19-bis delle slide).

`Scrittore`, `Disegnatore`

Stessi attributi (`nome`, `cognome`, `dataNascita` con `@Past`, `paese`) e un metodo `getNomeCompleto()` usato nei template (`${d.nomeCompleto}`: Thymeleaf chiama il getter). `Scrittore.fumetti` è una `@OneToMany(mappedBy = "scrittore")`; `Disegnatore.fumetti` è il lato inverso della molti-a-molti. Sono due classi separate perché la specifica le tratta come entità distinte.

> TEORIA
>
> 04-2 · sl. 2–10 **Ereditarietà (cenni).** Si poteva modellare una superclasse `Autore` con `@MappedSuperclass` (solo riuso di codice) oppure con una gerarchia `@Inheritance`. Il progetto non lo fa: è una scelta legittima, ma all'orale ti possono chiedere come lo faresti. `@MappedSuperclass` è la risposta più semplice, perché non serve fare query polimorfiche su "tutti gli autori".

`Recensione`

```
@Table(uniqueConstraints = @UniqueConstraint(name = "uk_recensione_figure_autore",
        columnNames = { "figure_id", "autore_id" }))
public class Recensione {
    @NotBlank private String titolo;
    @NotBlank @Size(max = 1000) @Column(length = 1000) private String testo;
    @NotNull @Min(1) @Max(5) private Integer voto;
    @Column(nullable = false) private LocalDate data;
    @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "autore_id") private User autore;
    @ManyToOne(fetch = FetchType.LAZY) @JoinColumn(name = "figure_id") private Figure figure;

    @PrePersist
    protected void onCreate() {
        if (data == null) data = LocalDate.now();
    }
}
```

- La regola "un utente, al massimo una recensione per figure" è dichiarata **due volte**: dal vincolo `UNIQUE(figure_id, autore_id)` nello schema e dal controllo nel service.
- `@PrePersist` è un *callback del ciclo di vita* JPA: Hibernate chiama `onCreate()` subito prima dell'INSERT, quindi ogni recensione nuova riceve la data di oggi senza che controller o service se ne occupino. Il `if (data == null)` evita di sovrascrivere una data impostata a mano. Le recensioni di esempio di `data.sql` non passano da JPA, quindi la loro data è scritta direttamente nello script.
- L'associazione verso `User` è **unidirezionale**: `User` non ha la lista delle sue recensioni, perché nessuna pagina la mostra.

`User` e `Credentials`

> TEORIA
>
> 10 · sl. 9–11 Il prof propone esattamente questa coppia: `User` con i dati anagrafici, `Credentials` con `username`, `password`, `role` e una `@OneToOne(cascade = ALL)` verso `User`; le costanti `DEFAULT_ROLE` e `ADMIN_ROLE`; `@Table(name = "users")` perché `user` è parola riservata in PostgreSQL.

FigureDB la ricalca fedelmente:

```
@Entity
public class Credentials {
    public static final String DEFAULT_ROLE = "DEFAULT";
    public static final String ADMIN_ROLE = "ADMIN";

    @NotBlank @Size(min = 3, max = 30) @Column(nullable = false, unique = true) private
String username;
    @NotBlank @Size(min = 3) @Column(nullable = false) private String password;
    @Column(nullable = false) private String role;

    @Valid
    @OneToOne(cascade = CascadeType.ALL)
    private User user;
}
```

- `cascade = ALL`: salvare le credenziali salva anche l'utente. Nella registrazione basta `credentialsRepository.save(credentials)`.
- `@Valid` **sul campo** `user`: dice a Bean Validation di validare **anche l'oggetto annidato**. Quando il controller valida `credentials`, controlla anche `@NotBlank` ed `@Email` di `user`: la form di registrazione,

che contiene entrambi, si valida con un solo `@Valid`.

- `User` ha `@Email` e `unique = true` sull'email; `getNomeCompleto()` serve alle recensioni.

> ATTENZIONE
>
> **`@Size(min = 3)` sulla password e BCrypt.** La validazione avviene sulla password **in chiaro** che arriva dalla form, prima che il controller la cifri. Dopo la cifratura la stringa è lunga 60 caratteri, quindi il vincolo resta soddisfatto. Va bene così, ma è utile sapere perché funziona.

Le enumerazioni

`Materiale` (PVC, ABS, PVC_ABS, RESINA, POLYSTONE, METALLO, VINILE, PORCELLANA) e `TipoSerie` (MANGA, ANIME, FUMETTO, GRAPHIC_NOVEL, VIDEOGIOCO, FILM, SERIE_TV, SPORT). Ogni costante ha una **etichetta** leggibile passata al costruttore (`PVC_ABS("PVC e ABS")`) e restituita da `getLabel()` : nel DB si salva il nome della costante, nelle pagine si mostra l'etichetta (`${m.label}`). Un enum Java può avere campi e costruttore come una classe: le costanti sono le sue uniche istanze.

> ALL'ORALE
>
> **Esercizio tipico: "collega le figure ai singoli personaggi"** (è nella lista delle estensioni della specifica). In `Figure` si aggiunge `@ManyToMany(fetch = LAZY) @JoinTable(name = "figure_personaggio") List<Personaggio> personaggi = new ArrayList<>()` : una figure può raffigurare più personaggi, e un personaggio compare in più figure. Se serve navigare al contrario, in `Personaggio` si aggiunge `@ManyToMany(mappedBy = "personaggi") List<Figure> figure` . Nel service si controlla che i personaggi appartengano alla serie della figure; nella form si usa una `<select multiple name="personaggiIds">` come per i disegnatori, e nel controller un `@RequestParam(required = false) List<Long> personaggiIds` .

4. Il Persistence Layer (package `repository`)

4.1 Il pattern Repository e `JpaRepository`

> TEORIA
>
> `03 · sl. 43–47`, `06 · sl. 3–8` Un repository "incapsula la logica di accesso al database" e si comporta "come una collezione persistente" (Fowler). In Spring basta dichiarare un'**interfaccia** che estende `CrudRepository<Entità, TipoId>`: il framework genera a runtime la classe che la implementa. `06 · sl. 7` I metodi ereditati: `save`, `findById` (restituisce `Optional`), `findAll`, `count`, `existsById`, `delete`, `deleteById` … `06 · sl. 10` `save()` fa INSERT se l'id è `null`, UPDATE altrimenti.

Ogni entità ha il suo repository: `AziendaRepository`, `CredentialsRepository`, `DisegnatoreRepository`, `EditoreRepository`, `FigureRepository`, `FumettoRepository`, `PersonaggioRepository`, `RecensioneRepository`, `ScrittoreRepository`, `SerieRepository`, `UserRepository`.

Tutti estendono **`JpaRepository`** invece di `CrudRepository`, come il progetto di riferimento. `JpaRepository` è una sotto-interfaccia di `CrudRepository`: eredita tutti i metodi CRUD e aggiunge `findAll(Sort)`, `findAll(Pageable)` e `findAll()` che restituisce `List` invece di `Iterable` (comodo, perché `List` si usa direttamente nei template e negli stream).

Nel progetto si usano tre tecniche per le query, in ordine crescente di controllo.

4.2 Tecnica 1: query derivate dal nome del metodo

> TEORIA
>
> `06 · sl. 12–15` "Il nome del metodo ne definisce la semantica: la query viene costruita a partire dal nome." Prefissi: `find`, `count`, `exists`, `delete`; condizioni combinate con `And`, `Or`; ordinamento con `OrderBy…Asc`. Il motivo per non fare `findAll()` e filtrare in Java è `06 · sl. 12`: "se ho milioni di prodotti saturo memoria e CPU! Queste operazioni le deve fare il DBMS".

Metodo	JPQL generata (circa)	Usato da
`List<Azienda> findAllByOrderByNomeAsc()`	`select a from Azienda a order by a.nome asc`	elenco aziende, tendine
`List<Scrittore> findAllByOrderByCognomeAscNomeAsc()`	`… order by s.cognome asc, s.nome asc`	elenco e tendina scrittori
`boolean existsByNomeIgnoreCase(String)`	`… where lower(a.nome) = lower(?)`	controllo duplicato azienda/serie
`Optional<Credentials> findByUsername(String)`	`… where c.username = ?`	login, utente corrente
`boolean existsByEmail(String)`	`… where u.email = ?`	registrazione
`boolean existsByFigureIdAndAutoreId(Long, Long)`	`… where r.figure.id = ? and r.autore.id = ?`	regola "una recensione"
`boolean existsByNomeIgnoreCaseAndAziendaIdAndDataUscita(…)`	tre condizioni in `and`	vincolo di consistenza delle figure

Metodo	JPQL generata (circa)	Usato da
`boolean existsByNomeIgnoreCaseAndSerieId(String, Long)`	`… where lower(p.nome)=lower(?) and p.serie.id=?`	personaggio duplicato
`List<Figure> findBySerieIdOrderByNomeAsc(Long)`	navigazione `serie.id`	dettaglio serie
`List<Fumetto> findBySerieIdOrderByAnnoAsc(Long)`	idem, ordinato per anno	dettaglio serie
`boolean existsByIsbn(String)`	`… where f.isbn = ?`	ISBN duplicato

Due osservazioni utili all'orale:

- **`SerieId` naviga l'associazione.** `findBySerieIdOrderByNomeAsc` non cerca un campo `serieId` (che non esiste): Spring Data spezza il nome in `serie` + `id` e genera `where f.serie.id = ?`. È la dot-notation di JPQL 05 · sl. 5.
- **Le `exists…` sono preferibili a "carica e controlla"**: il DB restituisce un solo booleano invece di un'entità intera.

4.3 Tecnica 2: `@Query` in JPQL con `JOIN FETCH`

> TEORIA
>
> 05 · sl. 2–6 **JPQL** è "simile all'SQL, ma con sintassi più vicina al mondo OO": si interrogano entità (con l'iniziale maiuscola), non tabelle, e si **navigano i riferimenti** con la dot-notation (`f.azienda.id`). JPA la traduce in SQL con le join necessarie. 05 · sl. 13–14 Parametri con `:nome` . 06 · sl. 16–17 `@Query("…")` associa una query a un metodo; `@Param("nome")` collega parametro del metodo e parametro della query. 06 · sl. 35 `JOIN FETCH` "forza l'ORM a caricare subito i dati collegati con un join"; con un semplice `JOIN` non li caricherebbe necessariamente.

Il progetto ha tre `@Query`:

```
// FigureRepository: ricerca per nome usata dalle API REST (widget React)
@Query("SELECT f FROM Figure f LEFT JOIN FETCH f.azienda LEFT JOIN FETCH f.serie "
        + "WHERE LOWER(f.nome) LIKE LOWER(CONCAT('%', :nome, '%')) ORDER BY f.nome")
List<Figure> cercaPerNome(@Param("nome") String nome);

// FigureRepository: scenario 2 del confronto sulle strategie di fetch (§11.2)
@Query("SELECT f FROM Figure f LEFT JOIN FETCH f.azienda ORDER BY f.nome")
List<Figure> findAllWithAzienda();

// RecensioneRepository: aggregazione calcolata dal DBMS
@Query("SELECT AVG(r.voto) FROM Recensione r WHERE r.figure.id = :figureId")
Double votoMedioPerFigure(@Param("figureId") Long figureId);
```

- **`LEFT JOIN FETCH`** carica azienda e serie **nella stessa query** della figure: il DTO della risposta REST usa `f.getAzienda().getNome()` e `f.getSerie().getNome()`, che quindi non scatenano altre SELECT. `LEFT` invece di `INNER` per prudenza: le colonne FK non sono `NOT NULL` nello schema, e una figure senza azienda non deve sparire dal risultato.
- **`LOWER(...) LIKE LOWER(CONCAT('%', :nome, '%'))`** realizza la ricerca "contiene", senza distinguere maiuscole e minuscole. La query derivata equivalente sarebbe `findByNomeContainingIgnoreCase`, che però non permette di aggiungere il fetch join.
- **`AVG(r.voto)`** è una funzione di aggregazione 05 · sl. 16–19: il DBMS calcola la media e restituisce un solo numero, invece di caricare tutte le recensioni e farla in Java. Restituisce `Double` (oggetto, non

`double`) perché per una figure senza recensioni la media è `NULL`: il template mostra il voto solo se `votoMedio != null`.

4.4 Tecnica 2-bis: `@EntityGraph`

> **TEORIA**
>
> **O6 · sl. 36** "In alternativa, possiamo forzare il metodo a eseguire il join: `@EntityGraph(attributePaths = "orders")`." Il risultato è lo stesso del `JOIN FETCH`, ma senza scrivere la query: si aggiunge l'annotazione a un metodo derivato.

È la tecnica più usata nel progetto, perché si combina con le query derivate:

```
// FigureRepository
@EntityGraph(attributePaths = { "azienda", "serie" })
Optional<Figure> findWithAziendaESerieById(Long id);            // dettaglio figure

@EntityGraph(attributePaths = { "azienda", "serie" })
Page<Figure> findAllBy(Pageable pageable);                      // elenco paginato

@EntityGraph(attributePaths = { "azienda" })
List<Figure> findAllBy();                                       // scenario 3 del confronto

// FumettoRepository
@EntityGraph(attributePaths = { "serie", "scrittore", "editore" })
List<Fumetto> findAllByOrderByTitoloAsc();                      // elenco fumetti

@EntityGraph(attributePaths = { "serie", "scrittore", "editore", "disegnatori" })
Optional<Fumetto> findCompletoById(Long id);                    // dettaglio fumetto

// RecensioneRepository
@EntityGraph(attributePaths = { "autore", "figure" })
List<Recensione> findAllByOrderByDataDesc();                    // pannello admin recensioni
```

Altri esempi: `SerieRepository.findWithPersonaggiById`, `AziendaRepository.findWithFigureById`, `ScrittoreRepository` / `DisegnatoreRepository` / `EditoreRepository.findWithFumettiById`, `RecensioneRepository.findByFigureIdOrderByDataDesc` (con `autore`).

Come leggerli:

- **I nomi `findWith…By…` e `findAllBy`.** Spring Data ignora le parole fra `find` e `By`: `findWithAziendaESerieById` è una normale `findById`, con un nome che ricorda *che cosa* viene caricato. `findAllBy()` è un `findAll` "con nome diverso": serve perché `findAll()` è già ereditato e non si può annotare in modo diverso senza cambiarne il comportamento ovunque.
- **`findCompletoById` carica una collezione (`disegnatori`).** Con una `@ManyToMany` il join produce una riga per disegnatore, ma per un singolo fumetto il risultato viene ricomposto in un solo oggetto.
- **Il pannello admin delle recensioni** carica autore e figure di tutte le recensioni con **una sola query**: senza grafo, la tabella farebbe 1 query per l'elenco + una per ogni autore + una per ogni figure (N+1).

La paginazione. `Page<Figure> findAllBy(Pageable pageable)` riceve un `PageRequest` (numero di pagina, dimensione, ordinamento) e Spring Data lo traduce in `LIMIT` / `OFFSET`: dal DB arrivano solo le 9 figure della pagina. Spring esegue anche una seconda query, `select count(*)`, per sapere quante pagine esistono. L'oggetto `Page` restituito contiene la lista (`getContent()`), il numero di pagina, il totale delle pagine e dei risultati, e i booleani `isFirst()` / `isLast()` usati dalla vista. Qui il grafo carica solo associazioni a singolo oggetto: se caricasse una collezione, Hibernate non potrebbe paginare nel DB e lo farebbe in memoria (warning `HHH90003004`).

ALL'ORALE

"Differenza tra `@EntityGraph` e `JOIN FETCH`?" Producono lo stesso SQL (un join). `JOIN FETCH` si scrive dentro una `@Query` JPQL, quindi serve quando la query ha condizioni complesse (come la ricerca con `LIKE` di `cercaPerNome`). `@EntityGraph` si aggiunge a un metodo derivato senza scrivere JPQL ed è riutilizzabile: è la scelta più comoda quando la condizione è semplice (`findById`, `findAll`, `findBySerieId`). La pagina `/admin/performance` li confronta: entrambi fanno 1 query (§11.2).

"Aggiungi una query che restituisce le figure di un materiale, ordinate per prezzo." `List<Figure> findByMaterialeOrderByPrezzoAsc(Materiale materiale);` con eventualmente `@EntityGraph(attributePaths = {"azienda", "serie"})` se la pagina mostra azienda e serie.

5. Il Service Layer (package `service`)

5.1 A che cosa serve

> TEORIA
>
> 03 · sl. 50–51 **Façade.** Le operazioni offerte dal sistema stanno in classi `@Service`, "una facciata attraverso la quale si accede ai servizi". 07 · sl. 53–54 Il Service Layer "implementa operazioni di business, coordina più repository, offre metodi che rappresentano i casi d'uso"; "i repository manipolano dati, i service implementano azioni", in modalità **transazionale**. 07 · sl. 53 Un service per entità *o aggregato di entità*.

FigureDB ha un service per ogni entità gestita dalle pagine: `FigureService`, `SerieService`, `FumettoService`, `AziendaService`, `ScrittoreService`, `DisegnatoreService`, `EditoreService`, `RecensioneService`, `CredentialsService`, `UserService`. A questi si aggiunge `FetchBenchmarkService`, che esegue l'analisi delle strategie di fetch (§11.2). `Personaggio` non ha un service proprio: fa parte dell'aggregato `Serie`, e lo gestisce `SerieService.aggiungiPersonaggio` 07 · sl. 53.

5.2 Iniezione delle dipendenze con `@Autowired`

```
@Service
public class FigureService {
    @Autowired
    private FigureRepository figureRepository;
    @Autowired
    private AziendaRepository aziendaRepository;
    @Autowired
    private SerieRepository serieRepository;
    ...
}
```

> TEORIA
>
> 03 · sl. 61, 80 "`@Autowired` permette di iniettare dipendenze": "ci pensa l'application server a creare un oggetto MovieRepository e ad assegnarlo alla variabile". È lo stile usato nelle slide e nel progetto di riferimento: annotazione direttamente sul campo.

Il funzionamento: all'avvio Spring crea **un solo oggetto** (singleton) per ogni classe `@Service`, `@Repository`, `@Controller`, e lo inserisce nei campi `@Autowired` di chi ne ha bisogno. Nessuna classe fa `new FigureRepository()`: la classe concreta del repository non esiste nemmeno nel codice, la genera Spring.

Alternativa da conoscere: l'iniezione via costruttore (usata nel progetto originale e, qui, in `SecurityConfiguration`). Si dichiara il campo `final` e lo si riceve nel costruttore: le dipendenze sono visibili nella firma e nei test si può costruire l'oggetto con `new` passando dei finti repository. Il comportamento a runtime è identico.

5.3 La gestione delle transazioni

> TEORIA
>
> 07 · sl. 7–15 **Proprietà ACID**: atomicità (tutto o niente), consistenza, isolamento, durabilità. 07 · sl. 16–38 **Anomalie**: perdita di aggiornamento, letture sporche, letture inconsistenti, fantasmi. 07 · sl. 42–45 **Livelli di isolamento**; PostgreSQL ne implementa tre, con **Read Committed di default**. 07 · sl. 55–56 In Spring `@Transactional` racchiude il corpo del metodo in una transazione; si possono indicare `isolation = Isolation.SERIALIZABLE` e `readOnly = true`. 07 · sl. 46–49 **Linee guida**: lettura non critica → Read Committed; "se il caso d'uso **legge dei dati per decidere se può scrivere** … va usato Serializable".

Le regole che FigureDB applica:

Tipo di metodo	Annotazione	Esempi	Motivo
Lettura	`@Transactional(readOnly = true)`	tutti i `find…`, `cercaPerNome`, `votoMedio`, `existsByEmail`	Nessun dirty checking a fine transazione; Hibernate e il driver possono ottimizzare
Scrittura **che dipende da un controllo**, caso di riferimento	`@Transactional(isolation = SERIALIZABLE)`	`FigureService.inserisciACatalogo`	Linea guida 07 · sl. 49: tra il controllo del duplicato e l'INSERT un'altra transazione potrebbe inserire la stessa figure
Scrittura	`@Transactional` (Read Committed)	`aggiorna`, `delete`, `save` di tutti i service, `RecensioneService.inserisci`	Read Committed è sufficiente 07 · sl. 47; il vincolo `UNIQUE` sul DB fa da ultima difesa

Come funziona `@Transactional` "dietro le quinte". Spring non chiama il tuo oggetto direttamente: lo avvolge in un **proxy**, lo stesso pattern che 06 · sl. 43–46 usa per spiegare il lazy loading. Il proxy apre la transazione, chiama il metodo vero, poi fa *commit* o, se esce una `RuntimeException`, *rollback*. Tutte le eccezioni di dominio estendono `RuntimeException`: un' `OperazioneNonValidaException` lanciata a metà operazione annulla automaticamente tutte le scritture già fatte (atomicità). Una conseguenza: chiamare un metodo `@Transactional` **dall'interno della stessa classe** (`this.altroMetodo()`) non passa dal proxy, quindi le sue impostazioni vengono ignorate e vale la transazione già aperta (propagazione `REQUIRED` 07 · sl. 59).

Che cosa succede con SERIALIZABLE su PostgreSQL. PostgreSQL non blocca tutto. Esegue le transazioni in parallelo e, se rileva che il risultato non sarebbe ottenibile con un'esecuzione seriale, ne fa **fallire una** al commit con un errore di serializzazione (SQLSTATE `40001`). Due inserimenti concorrenti della stessa figure non possono quindi riuscire entrambi. Il client della transazione fallita vede un errore 500 (lo gestisce `handleUnexpectedException`, §6.4), a meno che l'operazione non venga ritentata.

5.4 Le eccezioni di dominio

> TEORIA
>
> 08 · sl. 40–46, 56 **Validazione delle regole di business.** "Le regole di business vanno verificate nel Service. In caso di violazioni, il service solleva una opportuna eccezione." 08 · sl. 61–64 Per la risorsa inesistente: un'eccezione `…NotFoundException extends RuntimeException` tradotta in una pagina 404. 08 · sl. 67–71 Gli errori di dominio passano da un gestore centralizzato `@ControllerAdvice`.

Le tre eccezioni stanno nel package `service`, accanto ai service che le lanciano:

Eccezione	Significato	Pagina e codice HTTP (§6.4)
`RisorsaNonTrovataException`	l'id richiesto non esiste	`error/404`, **404 Not Found**
`OperazioneNonValidaException`	vincolo di consistenza violato: figure duplicata, ISBN già usato, nome già esistente, cancellazione di una serie/azienda/autore con entità collegate	`error/errore`, **409 Conflict**
`RecensioneDuplicataException`	l'utente ha già recensito quella figure	`error/errore`, **409 Conflict**

Per i controlli di **proprietà** (modificare o cancellare la recensione di un altro) si usa invece `AccessDeniedException` di Spring Security, tradotta in **403 Forbidden**.

Lo schema ricorrente per la ricerca per id:

```
Azienda azienda = aziendaRepository.findById(aziendaId)
        .orElseThrow(() -> new RisorsaNonTrovataException("l'azienda", aziendaId));
```

`orElseThrow` sull'`Optional` sostituisce l'`if (optional.isPresent()) … else throw` della slide. Il costruttore a due argomenti compone il messaggio "Non esiste l'azienda con id 42.".

5.5 I service uno per uno

`FigureService`: il caso d'uso multi-entità

I metodi di lettura (`findById`, `findByIdWithAziendaESerie`, `findAll`, `findPagina`, `cercaPerNome`, `findBySerieId`, `findByAziendaId`) sono `readOnly` e delegano ciascuno a un metodo del repository. `findPagina` costruisce la richiesta di pagina:

```
return figureRepository.findAllBy(
        PageRequest.of(numeroPagina, figurePerPagina, Sort.by("nome").ascending()));
```

L'ordinamento è indispensabile: senza un ordine stabile il DB potrebbe restituire le righe in ordine diverso da una richiesta all'altra, e una figure potrebbe comparire in due pagine o in nessuna.

`inserisciACatalogo` è l'operazione di riferimento della specifica (§7) e il punto più importante da saper raccontare:

```java
@Transactional(isolation = Isolation.SERIALIZABLE)
public Figure inserisciACatalogo(Figure figure, Long aziendaId, Long serieId) {
    Azienda azienda = aziendaRepository.findById(aziendaId)                          // 1
            .orElseThrow(() -> new RisorsaNonTrovataException("l'azienda", aziendaId));
    Serie serie = serieRepository.findById(serieId)                                  // 2
            .orElseThrow(() -> new RisorsaNonTrovataException("la serie", serieId));
    if (figureRepository.existsByNomeIgnoreCaseAndAziendaIdAndDataUscita(            // 3
            figure.getNome(), aziendaId, figure.getDataUscita())) {
        logger.warn("Figure duplicata rifiutata: {} di {} del {}", ...);
        throw new OperazioneNonValidaException("L'azienda ... ha gia' a catalogo ...");
    }
    figure.setAzienda(azienda);                                                      // 4-5
    figure.setSerie(serie);
    Figure salvata = figureRepository.save(figure);
    logger.info("Figure inserita a catalogo: {} ({}, serie {})", ...);
    return salvata;
}
```

Coinvolge tre repository in un'unica transazione atomica: se il passo 3 fallisce non viene scritto niente. È l'equivalente del `payOrder` della slide `07 · sl. 57`: verifica, poi scrittura, con SERIALIZABLE perché la decisione di scrivere dipende da una lettura. Basta impostare il lato proprietario (`figure.setAzienda`), perché la FK sta nella tabella `figure` .

TEORIA

`08 · sl. 76–87` **Logging.** Le stampe con `System.out.println` sono scomode: non si possono filtrare né spegnere senza ricompilare. Si usa un `Logger` SLF4J (`LoggerFactory.getLogger(Classe.class)`) con livelli (`debug` , `info` , `warn` , `error`) e segnaposto `{}` . `FigureService` registra con `info` ogni inserimento riuscito e con `warn` ogni duplicato rifiutato; `GlobalExceptionHandler` registra con `error` gli errori inattesi, stack trace compreso.

`aggiorna` usa uno schema che ritorna in diversi service: **non** si salva l'oggetto arrivato dalla form, ma si **copiano i campi** sull'entità *gestita* caricata dal DB e si salva quella:

```java
Figure figure = figureRepository.findById(id).orElseThrow(...);
figure.setNome(dati.getNome());
figure.setDataUscita(dati.getDataUscita());
... // altezza, materiale, prezzo, descrizione, immagine, azienda, serie
return figureRepository.save(figure);
```

Il commento nel codice spiega il rischio concreto: l'oggetto della form ha la lista `recensioni` vuota e, dato che `Figure.recensioni` ha `orphanRemoval = true` , un `save` diretto (che diventa un *merge*) **cancellerebbe tutte le recensioni**. `SerieService.aggiorna` ha lo stesso problema con i personaggi, `AziendaService.aggiorna` e `FumettoService.aggiorna` usano lo stesso schema per prudenza.

`delete` : `figureRepository.deleteById(id)` . Le recensioni vengono cancellate in cascata (`CascadeType.REMOVE`).

`RecensioneService` : le regole di proprietà

- **`inserisci(figureId, autore, titolo, testo, voto)`** : se `autore` è `null` lancia `AccessDeniedException` ; recupera la figure (`RisorsaNonTrovataException` se manca); se `existsByFigureIdAndAutoreId` è vero lancia `RecensioneDuplicataException` ; altrimenti crea la recensione, collega figure e autore e salva. L'autore arriva dal controller, che lo ricava da Spring Security: **non** dalla form, altrimenti un utente potrebbe recensire a nome di un altro.

- `modifica(id, titolo, testo, voto, utente)` : solo l'autore può modificare, nemmeno l'ADMIN. Il confronto è fra gli **id** degli utenti (`Objects.equals(recensione.getAutore().getId(), utente.getId())`). Se il voto non arriva (`null`) resta quello vecchio.
- `elimina(id, utente, isAdmin)` : l'autore oppure l'ADMIN (moderazione). Se nessuna delle due condizioni vale, `AccessDeniedException` .
- `findAll()` : tutte le recensioni del sito, dalla più recente, con autore e figure già caricati; alimenta il pannello `/admin/recensioni` .
- `votoMedio(figureId)` : delega alla `@Query` con `AVG` .

`SerieService`

- `save` controlla il nome duplicato **solo in inserimento** (`serie.getId() == null && existsByNomeIgnoreCase(...)`).
- `aggiorna` copia nome, tipo, descrizione e logo sull'entità gestita: un save diretto cancellerebbe i personaggi (orphanRemoval).
- `aggiungiPersonaggio(serieId, nome, descrizione)` : coinvolge due entità nella stessa transazione. Recupera la serie, controlla che il nome non sia vuoto e che non esista già nella stessa serie (`existsByNomeIgnoreCaseAndSerieId`), poi crea e salva il personaggio.
- `delete` : carica la serie e, se `getFumetti()` o `getFigure()` non sono vuote, lancia `OperazioneNonValidaException` . È il vincolo "non si può eliminare una serie a cui sono collegate altre entità". Le liste LAZY vengono inizializzate qui, dentro la transazione.

`FumettoService`

`inserisci` coinvolge **cinque** repository (`Fumetto` , `Serie` , `Scrittore` , `Editore` , `Disegnatore`): controlla che l'ISBN non sia già usato, poi il metodo privato `collega(...)` recupera serie, scrittore ed editore per id e i disegnatori con `disegnatoreRepository.findAllById(disegnatoriIds)` . Se la lista dei disegnatori è vuota lancia `OperazioneNonValidaException` : è la regola "un fumetto ha almeno un disegnatore", cioè la cardinalità `1..*` che lo schema relazionale non riesce a esprimere. `aggiorna` controlla l'ISBN solo se è cambiato (`!fumetto.getIsbn().equals(dati.getIsbn())`), così il fumetto non risulta duplicato di sé stesso.

`AziendaService` , `ScrittoreService` , `DisegnatoreService` , `EditoreService`

Seguono lo stesso schema: `find…` readOnly, `findAll` ordinato, `findByIdWith…` con `@EntityGraph` per il dettaglio, `delete` con controllo delle entità collegate (`AziendaService` : figure; gli altri tre: fumetti). `AziendaService` controlla anche il nome duplicato in `save` e ha un `aggiorna` che copia i campi.

Scrittori, disegnatori ed editori hanno un `save` semplice, usato sia per l'inserimento sia per la modifica: il controller imposta l'id preso dall'URL (`scrittore.setId(id)`) e chiama `save` , che diventa un UPDATE. Qui il save diretto è sicuro: la lista `fumetti` è il **lato inverso** senza cascade né orphanRemoval, e JPA ignora il lato inverso quando scrive. È un buon esempio da citare per spiegare *perché* `Serie` e `Figure` invece richiedono la copia dei campi.

`CredentialsService` e `UserService`

Sottili strati sopra i repository: `getCredentials(username)` (usato da login, profilo, registrazione), `saveCredentials` (salva anche l'utente per la cascata), `getUserByUsername(username)` (usato per ricavare l'utente corrente), `existsByEmail` . La logica della registrazione (controlli di unicità, cifratura della password, ruolo DEFAULT) sta nel controller, come nel progetto di riferimento (§6.3).

`FetchBenchmarkService`

È descritto al §11.2, insieme alla pagina `/admin/performance` .

6. Il Controller Layer MVC (package `controller`)

6.1 Che cosa fa un controller

> **TEORIA**
>
> 08 · sl. 4–6 Il controller "gestisce le richieste HTTP, valida e converte i dati, esegue un'operazione sul Service Layer, decide quale vista produrrà la risposta". 03 · sl. 56–60 Si scrive un metodo per ogni richiesta (metodo HTTP + URL), annotato `@GetMapping` / `@PostMapping` ; i dati per la vista vanno nel `Model` con `addAttribute` ; la stringa restituita è il nome del template.

Le annotazioni che ricorrono nei controller di FigureDB:

Annotazione	Significato	Esempio
`@Controller`	classe che gestisce richieste e restituisce viste	tutti i controller MVC
`@GetMapping` , `@PostMapping`	associa metodo e richiesta	`@GetMapping("/figure/{id}")`
`@PathVariable Long id`	estrae `{id}` dall'URL e lo converte in `Long` 03 · sl. 63	`/figure/7` → `id = 7`
`@RequestParam`	parametro della query string o campo di form non mappato su un oggetto 08 · sl. 98	`?page=2` , `aziendaId` , `disegnatoriIds`
`@ModelAttribute("figureForm") Figure`	*binding*: crea l'oggetto dai campi della form 08 · sl. 19	form admin
`@Valid` + `BindingResult`	valida l'oggetto e raccoglie gli errori 08 · sl. 20	tutte le POST con form
`RedirectAttributes`	attributi *flash* che sopravvivono a un redirect	errori delle recensioni

Gli URL seguono uno schema fisso, lo stesso del progetto di riferimento:

URL	Metodo	Vista	Chi
`/figure` , `/serie` , `/fumetti` , `/aziende`	GET	`listX`	tutti
`/figure/{id}` , `/serie/{id}` , … `/scrittori/{id}` , `/disegnatori/{id}` , `/editori/{id}`	GET	`showX`	tutti
`/admin/X/new`	GET	`formX` (vuota)	ADMIN
`/admin/X`	POST	redirect al dettaglio	ADMIN
`/admin/X/{id}/edit`	GET / POST	`formX` (precompilata) / redirect	ADMIN
`/admin/X/{id}/delete`	POST	redirect all'elenco	ADMIN

Tutto ciò che modifica dati è sotto `/admin/**` (tranne recensioni e registrazione), quindi una sola regola in `SecurityConfiguration` protegge tutte le operazioni dell'amministratore (§8.3).

6.2 Lo schema delle POST con form

Tutte le operazioni di scrittura seguono questo schema, preso da `FigureController.newFigure`:

```
@PostMapping("/admin/figure")
public String newFigure(@Valid @ModelAttribute("figureForm") Figure figureForm,
        BindingResult bindingResult,                        // SUBITO dopo l'oggetto
        @RequestParam Long aziendaId,
        @RequestParam Long serieId,
        Model model) {
    if (bindingResult.hasErrors()) {                        // (a) errori di input
        aggiungiDatiForm(model);
        return "formFigure";
    }
    try {
        Figure salvata = figureService.inserisciACatalogo(figureForm, aziendaId, serieId);
        return "redirect:/figure/" + salvata.getId();       // (c) Post/Redirect/Get
    } catch (OperazioneNonValidaException e) {              // (b) errore di business
        model.addAttribute("errore", e.getMessage());
        aggiungiDatiForm(model);
        return "formFigure";
    }
}
```

TEORIA

08 · sl. 20 "ATTENZIONE ALL'ORDINE DEI PARAMETRI: BindingResult va subito dopo l'oggetto che stiamo creando, prima di Model." 08 · sl. 45, 56 Se il service lancia un'eccezione di business, il controller la cattura e torna alla form con un messaggio. 03-2 · sl. 13–16 **Redirect:** dopo una POST riuscita si risponde con `302` (`return "redirect:…"`), così il browser fa una GET: se l'utente ricarica la pagina non reinvia la POST, e la figure non viene inserita due volte.

Tre esiti possibili:

- **(a) Errori di binding o validazione.** Si torna alla form *senza redirect*: l'oggetto `figureForm` con i dati inseriti e il `BindingResult` sono ancora nel model, quindi la form si ripresenta precompilata con gli errori accanto ai campi. Le tendine (aziende, serie, materiali) vanno **ripopolate**, perché il model è nuovo a ogni richiesta: per questo esiste il metodo privato `aggiungiDatiForm(model)`.
- **(b) Violazione di una regola di business.** Il messaggio va nell'attributo `errore` e si ripresenta la form, che lo mostra in alto (`<p th:if="${errore}" …>`).
- **(c) Successo.** Redirect al dettaglio della figure appena creata.

`aziendaId` e `serieId` arrivano come `@RequestParam` e non dentro `Figure`, perché nella form sono semplici `<select name="aziendaId">` e non campi dell'oggetto. Il service usa gli id per caricare le entità vere: il controller non deve conoscere i repository.

Perché l'attributo si chiama `figureForm` e non `figure`? Nel dettaglio, `figure` è la figure mostrata; nella form serve un nome diverso per non confondere i due usi. Il nome dell'attributo è anche quello che compare nelle chiavi di `message.properties` (`NotBlank.figureForm.nome`, §2.3).

6.3 I controller uno per uno

`HomeController` e `PerformanceController`

`GET /` → `index` (la home con i due bottoni "Vedi le Figure" e "Vedi le Serie"); `GET /admin/dashboard` → `adminDashboard` (le card verso tutte le sezioni di gestione). `PerformanceController` espone `GET`

`/admin/performance`: chiama `FetchBenchmarkService.eseguiConfronto()`, calcola il tempo massimo (per la larghezza delle barre) e mette nel model `risultati`, `numeroFigure`, `numeroAziende`, `maxTempoMs` (§11.2).

`FigureController`

- **`GET /figure?page=N`**: elenco paginato, 9 figure per pagina (costante `FIGURE_PER_PAGINA`). `@RequestParam(defaultValue = "0") int page`: se il parametro manca si intende la prima pagina. `Math.max(page, 0)` neutralizza un numero negativo; se si chiede una pagina oltre l'ultima, si fa redirect all'ultima esistente. Nel model vanno `figure` (la lista della pagina) e `pagina` (l'oggetto `Page`, per la barra di navigazione).
- **`GET /figure/cerca`**: restituisce `cercaFigure`, la pagina che ospita il widget React (§10).
- **`GET /figure/{id}`**: dettaglio con `figure`, `recensioni` e `votoMedio`; `RisorsaNonTrovataException` se l'id non esiste.
- **Admin**: `GET /admin/figure/new`, `POST /admin/figure`, `GET` e `POST /admin/figure/{id}/edit`, `POST /admin/figure/{id}/delete`. Inserimento e modifica usano lo stesso template `formFigure`, che decide titolo e URL d'invio in base a `figureForm.id` (nullo = nuova).

`SerieController`, `FumettoController`, `AziendaController`

Seguono lo stesso schema `list` / `show` / form admin.

- `SerieController.show` è il caso d'uso "il dettaglio della serie dà accesso a fumetti, figure e personaggi": mette nel model `serie` (con i personaggi caricati da `@EntityGraph`), `fumetti` e `figure` (due query separate). Ha in più `POST /admin/serie/{id}/personaggi`, che riceve `nome` e `descrizione` come `@RequestParam` dalla piccola form nel dettaglio e chiama `serieService.aggiungiPersonaggio`.
- `FumettoController` riceve dalla form `serieId`, `scrittoreId`, `editoreId` e la lista **`@RequestParam(required = false) List<Long> disegnatoriIds`**: la `<select multiple>` invia più parametri con lo stesso nome e Spring li raccoglie in una lista `08 · sl. 97–98`. `required = false` evita un errore 400 se non se ne seleziona nessuno: in quel caso il service lancia `OperazioneNonValidaException` con un messaggio chiaro.
- `AziendaController` cattura l'`OperazioneNonValidaException` del nome duplicato e la mostra nella form.

Le cancellazioni (`delete`) **non** catturano l'eccezione: se si prova a eliminare una serie con figure collegate, l'`OperazioneNonValidaException` risale fino a `GlobalExceptionHandler`, che mostra `error/errore.html` con il messaggio e codice 409.

`ScrittoreController`, `DisegnatoreController`, `EditoreController`

Il dettaglio è pubblico (`/scrittori/{id}`, raggiungibile dal dettaglio di un fumetto); elenco e form stanno sotto `/admin` (`/admin/scrittori` → `listScrittori`, una tabella con nome, paese, numero di fumetti e bottoni). In modifica il controller fa `scrittore.setId(id)` e chiama `save`, che diventa un UPDATE (§5.5).

`RecensioneController` (utenti registrati e admin)

- **`POST /figure/{figureId}/recensione`**: riceve `titolo`, `testo` e `voto` come `@RequestParam` e chiama `recensioneService.inserisci(figureId, utenteCorrente(), …)`. Se l'utente ha già recensito, cattura `RecensioneDuplicataException` e passa il messaggio come **flash attribute** (`redirectAttrs.addFlashAttribute("errore", …)`): Spring lo salva in sessione per un'unica richiesta successiva, la GET del redirect, e il dettaglio della figure lo mostra. Con un normale `Model` il messaggio andrebbe perso nel redirect.
- **`POST /figure/{figureId}/recensione/{id}/edit`** e **`…/delete`**: delegano al service; se l'utente non è l'autore (né admin, per la cancellazione) catturano `AccessDeniedException` e mostrano un messaggio flash.

- **`GET /admin/recensioni`** → `listRecensioni`: il pannello di moderazione con **tutte le recensioni del sito** (figure, autore, titolo, voto, data, testo). **`POST /admin/recensioni/{id}/delete`** cancella qualunque recensione (`isAdmin = true`).
- Due metodi privati: `utenteCorrente()` ricava lo username da `SecurityContextHolder` e lo trasforma nell'entità `User`; `isAdmin()` controlla se fra le *authorities* c'è `ROLE_ADMIN`.

`AuthenticationController`

`GET /login` mostra la form; la `POST /login` **non** è scritta nel controller, perché la gestisce direttamente Spring Security (§8.4). `GET /profilo` mostra i dati dell'utente loggato.

`GET /register` mette nel model un oggetto `Credentials` che contiene già un `User` vuoto; la form usa `th:field="*{user.nome}"` per i campi annidati. `POST /register`:

```
@PostMapping("/register")
public String registerUser(@Valid @ModelAttribute("credentials") Credentials credentials,
        BindingResult bindingResult) {
    if (bindingResult.hasErrors()) return "register";
    if (credentialsService.getCredentials(credentials.getUsername()).isPresent()) {
        bindingResult.rejectValue("username", "duplicate.username", "Username gia' in
uso...");
        return "register";
    }
    if (userService.existsByEmail(credentials.getUser().getEmail())) {
        bindingResult.rejectValue("user.email", "duplicate.email", "Email gia' registrata.");
        return "register";
    }
    credentials.setPassword(passwordEncoder.encode(credentials.getPassword()));
    credentials.setRole(Credentials.DEFAULT_ROLE);
    credentialsService.saveCredentials(credentials);       // salva anche User (cascade ALL)
    return "redirect:/login?registered=true";
}
```

TEORIA

10 · sl. 22 "Gli utenti vengono registrati con il ruolo DEFAULT: sarebbe poco sensato permettere di creare utenze con il ruolo di amministratore." La password va cifrata con lo stesso `PasswordEncoder` (BCrypt) usato al login. 08 · sl. 56 "Il controller cattura l'eccezione e la gestisce registrando un errore su BindingResult, che verrà mostrato dal template": qui `rejectValue("username", …)` attacca l'errore al campo, che compare sotto l'input come gli errori di validazione.

- **Un solo `@Valid`** valida anche lo `User` annidato grazie al `@Valid` sul campo `Credentials.user` (§3.4).
- **Il ruolo è imposto dal controller** e non letto dalla form: anche se un utente aggiungesse a mano un campo `role=ADMIN` nella richiesta, il valore verrebbe sovrascritto.
- **`redirect:/login?registered=true`**: la pagina di login mostra "Registrazione completata" leggendo `${param.registered}`.

6.4 I tre `@ControllerAdvice`

Un `@ControllerAdvice` è una classe i cui metodi si applicano **a tutti i controller**. Il progetto ne ha tre, ciascuno con un compito.

`GlobalBindingConfig`

```java
@InitBinder
public void initBinder(WebDataBinder binder) {
    binder.registerCustomEditor(String.class, new StringTrimmerEditor(true));
}
```

> **TEORIA**
>
> 08 · sl. 95–96 `StringTrimmerEditor(true)` in un `@InitBinder` globale toglie gli spazi iniziali e finali da ogni stringa delle form; il `true` trasforma la stringa vuota in `null`, così `@NotBlank` / `@NotNull` scattano correttamente e `" Goku "` non finisce nel DB con gli spazi.

`GlobalController`

```java
@ModelAttribute("userDetails")
public UserDetails getUser() {
    Authentication auth = SecurityContextHolder.getContext().getAuthentication();
    if (auth == null || auth instanceof AnonymousAuthenticationToken) return null;
    Object principal = auth.getPrincipal();
    return (principal instanceof UserDetails ud) ? ud : null;
}

@ModelAttribute("currentUserId")
public Long getCurrentUserId() {
    ... User u = credentialsService.getUserByUsername(auth.getName());
    return u != null ? u.getId() : null;
}
```

> **TEORIA**
>
> 10 · sl. 20–21 Un `@ModelAttribute` in un `@ControllerAdvice` rende disponibile `userDetails` a **tutti** i template; l'utente si legge da `SecurityContextHolder`, scartando l' `AnonymousAuthenticationToken` (l'"utente anonimo" che Spring Security usa per chi non ha fatto login).

FigureDB aggiunge `currentUserId`, l'id dell'utente loggato: `showFigure.html` lo confronta con `recensione.autore.id` per mostrare il bottone "Modifica" solo sulle proprie recensioni. Il *pattern matching* `principal instanceof UserDetails ud` (Java 16+) unisce il controllo del tipo e il cast.

`GlobalExceptionHandler`

```java
@ExceptionHandler(RisorsaNonTrovataException.class)
@ResponseStatus(HttpStatus.NOT_FOUND)
public String handleRisorsaNonTrovata(RisorsaNonTrovataException e, Model model) {
    logger.warn("Risorsa non trovata: {}", e.getMessage());
    model.addAttribute("errorMessage", e.getMessage());
    return "error/404";
}
// NoResourceFoundException        -> 404, error/404     (URL non mappato)
// OperazioneNonValidaException    -> 409, error/errore
// RecensioneDuplicataException    -> 409, error/errore
// AccessDeniedException           -> 403, error/errore
// Exception (tutto il resto)      -> 500, error/500     + logger.error con lo stack trace
```

TEORIA

08 · sl. 58–59 La *Whitelabel Error Page* compare quando un URL non è mappato (404), per errori di dominio (409) e per eccezioni non gestite (500); all'utente va dato un messaggio comprensibile, a chi mantiene l'applicazione una traccia nei log. 08 · sl. 61–66 Due 404 diversi: *la risorsa applicativa non esiste* (eccezione del service) e *l'URL non esiste proprio* (`NoResourceFoundException`). 08 · sl. 67–71 `@ControllerAdvice` + `@ExceptionHandler` + `@ResponseStatus` : "separa le responsabilità, evita try/catch nei controller, centralizza la gestione". 08 · sl. 72–74 Un metodo per le eccezioni generiche e la pagina `500.html` .

La classe ricalca le slide quasi riga per riga, compresi i nomi `handleNoResourceFound` , `handleUnexpectedException` e l'attributo `errorMessage` . I template stanno in `templates/error/` : `404.html` , `500.html` ed `errore.html` (per 409 e 403).

L'ordine conta poco, la specificità sì. Quando parte un'eccezione Spring sceglie l'handler con il tipo **più specifico**: una `RisorsaNonTrovataException` va al suo handler e non a quello generico su `Exception` , anche se entrambi sarebbero compatibili.

ATTENZIONE

L' `AccessDeniedException` lanciata dai filtri non arriva qui. L'handler 403 intercetta solo le eccezioni lanciate *dentro* un controller o un service (per esempio dai controlli di proprietà delle recensioni, che comunque il controller cattura prima). Quando è Spring Security a bloccare la richiesta, cioè un utente DEFAULT che apre `/admin/dashboard` , l'eccezione nasce nella catena di filtri, prima di qualsiasi controller: il browser riceve un 403 con la *Whitelabel Error Page*. Rimedio in una riga: creare `templates/error/403.html` , che Spring Boot usa automaticamente per quel codice (§14).

7. Le viste Thymeleaf e il CSS

7.1 Thymeleaf: i concetti usati

> TEORIA
>
> 03 · sl. 67–75 Thymeleaf è un motore di *natural templates*: espressioni e istruzioni stanno in attributi `th:*`, quindi il file resta un HTML valido apribile nel browser, con testo fittizio che verrà sostituito.
> 08 · sl. 52 **Le espressioni**: `${…}` variabili del model, `*{…}` campi dell'oggetto selezionato con `th:object`, `#{…}` messaggi i18n, `@{…}` URL, `~{…}` frammenti.

Costrutto	Dove nel progetto	Che cosa fa
`th:text="${figure.nome}"`	ovunque	sostituisce il testo (con escape HTML: niente XSS)
`th:each="f : ${figure}"`	elenchi	ciclo 03 · sl. 73
`th:if` / `th:unless`	liste vuote, messaggi	condizioni 03 · sl. 74
`@{'/figure/' + ${f.id}}`	link	URL costruito concatenando stringhe, con context path automatico
`th:object` + `th:field="*{nome}"`	form	binding bidirezionale 03-2 · sl. 8
`${#fields.hasErrors('nome')}`, `th:errors`	form	errori di validazione 08 · sl. 22
`th:replace="~{fragments/navbar :: navbar}"`	ogni pagina	riuso di frammenti 08 · sl. 88–91
`#temporals.format`, `#numbers.formatDecimal`, `#numbers.sequence`	dettagli, paginazione	*utility objects* di Thymeleaf
`th:classappend`, `th:style`, `th:selected`	paginazione, grafico prestazioni, tendine	attributi dinamici
`${param.error}`	`login.html`	parametro `?error=true` della query string
`${r.autore?.nomeCompleto ?: 'Anonimo'}`	recensioni	*safe navigation* (`?.`) ed *Elvis* (`?:`): niente errore se l'autore è nullo
`sec:authorize`, `sec:authentication`	navbar, bottoni admin	dialetto Spring Security (§7.3)

I conteggi con `.size()`. Nelle pagine i conteggi si fanno chiamando `size()` direttamente sulle collezioni delle entità: `${s.figure.size()}` in `listSerie`, `${azienda.figure.size()}` in `showAzienda`, `${x.fumetti.size()}` in `listScrittori`, `${recensioni.size()}` in `showFigure`. Funziona perché `open-in-view=true` tiene aperta la sessione durante il rendering (§2.2). Nei template si usa la sintassi dei metodi Java (`size()`, `isEmpty()`) perché sono le liste Java dell'entità; `figure.isEmpty()` si poteva scrivere anche `#lists.isEmpty(figure)`.

7.2 Il frammento `navbar`

> TEORIA
>
> 08 · sl. 88–91 **Fragments:** `th:fragment="nome"` definisce un pezzo di pagina riusabile; le pagine lo includono con `th:replace="~{file :: nome}"`.

Ogni pagina ha lo stesso `<head>` (titolo `FigureDB - …`, meta viewport, link a `stile.css`) e include la barra di navigazione con `<div th:replace="~{fragments/navbar :: navbar}"></div>`. Il frammento contiene il logo, i link pubblici (Figure, Serie, Fumetti, Aziende, Figure React) e una parte che cambia in base all'utente:

```
<a sec:authorize="hasRole('ADMIN')" th:href="@{/admin/dashboard}" class="btn-admin">Admin</a>
<a sec:authorize="isAuthenticated()" th:href="@{/profilo}"><span
sec:authentication="name">utente</span></a>
<form sec:authorize="isAuthenticated()" th:action="@{/logout}" method="POST">
    <button type="submit" class="btn-logout">Esci</button>
</form>
<a sec:authorize="!isAuthenticated()" th:href="@{/login}">Accedi</a>
<a sec:authorize="!isAuthenticated()" th:href="@{/register}">Registrati</a>
```

Il **logout è una form POST**, non un link, perché in Spring Security `/logout` accetta solo POST quando CSRF è attivo 10 · sl. 19.

7.3 Il dialetto `sec:` di Spring Security

`thymeleaf-extras-springsecurity6` aggiunge ai template attributi che interrogano Spring Security:

- **`sec:authorize="espressione"`** mostra l'elemento solo se l'espressione è vera: `isAuthenticated()`, `!isAuthenticated()`, `hasRole('ADMIN')`. Sono le stesse espressioni della `SecurityConfiguration`.
- **`sec:authentication="name"`** stampa lo username dell'utente loggato.
- **`#authorization.expression('hasRole(''ADMIN'')')`** è la stessa cosa usata *dentro* un'espressione `th:if`, per combinarla con altre condizioni. In `showFigure.html` i bottoni di una recensione compaiono se `recensione.autore.id == currentUserId` **oppure** l'utente è ADMIN; gli apici doppi `''` sono l'escape dell'apice dentro la stringa.

Nascondere i bottoni ai non-admin è **solo una comodità dell'interfaccia**: la protezione vera è nella `SecurityConfiguration` (URL `/admin/**`) e nel service (proprietà delle recensioni). Un utente che invia a mano una POST a `/admin/figure/3/delete` riceve comunque 403.

7.4 Le form e la protezione CSRF

In `formFigure.html`:

```
<form th:action="${figureForm.id != null ? '/admin/figure/' + figureForm.id + '/edit' :
'/admin/figure'}"
      method="POST" th:object="${figureForm}">
    <input required type="text" id="nome" th:field="*{nome}" />
    <span th:if="${#fields.hasErrors('nome')}" th:errors="*{nome}" class="error-msg"></span>

    <select name="aziendaId" id="aziendaId" required>
        <option th:each="a : ${aziende}" th:value="${a.id}" th:text="${a.nome}"
                th:selected="${figureForm.azienda != null and figureForm.azienda.id ==
a.id}">Azienda</option>
    </select>

    <select id="materiale" th:field="*{materiale}" required>
        <option th:each="m : ${materiali}" th:value="${m}"
th:text="${m.label}">Materiale</option>
    </select>
    ...
</form>
```

- **Una form per due casi d'uso.** L'operatore ternario in `th:action` sceglie l'URL: `/admin/figure/5/edit` se `figureForm` ha un id (modifica), `/admin/figure` altrimenti (inserimento). Anche il titolo `<h2>` usa lo stesso trucco.
- `th:field="*{nome}"` genera `id`, `name="nome"` e `value` precompilato: al ritorno dopo un errore la form non si svuota.
- `required`, `min`, `step` sono **validazione lato client** HTML5 08 · sl. 9; il server ricontrolla con Bean Validation 08 · sl. 10.
- **Le tendine di azienda e serie** non usano `th:field` (non sono campi di `Figure`) ma `name="aziendaId"`; `th:selected` preseleziona il valore corrente in modifica. La tendina del materiale invece usa `th:field="*{materiale}"`: è un campo dell'entità, e Spring converte la stringa `"PVC"` nella costante dell'enum.
- **La select multipla dei disegnatori** (`formFumetto.html`): `<select name="disegnatoriIds" multiple>` con `th:selected="${fumettoForm.disegnatori.contains(d)}"`. `contains` usa `equals`, che confronta nome, cognome e data di nascita (§3.1).
- Usando `th:action`, Thymeleaf aggiunge automaticamente il campo nascosto `_csrf` con il token 10 · sl. 30. Tutte le form del progetto usano `th:action`, e per questo funzionano con CSRF attivo.

7.5 Le altre pagine

- **`listFigure.html`**: griglia di card (immagine, nome, azienda e serie, prezzo) e barra di paginazione. I numeri di pagina si generano con `th:each="i : ${#numbers.sequence(0, pagina.totalPages - 1)}"`; la pagina corrente riceve una classe in più con `th:classappend="${i == pagina.number} ? 'pagination-current' : ''"`; "Precedente" e "Successiva" sono disattivati con `pagina.first` e `pagina.last`. Sotto: "Pagina 1 di 3 - 20 figure in totale". Il numero di pagina sta nell'URL (`?page=N`), quindi ogni pagina ha un indirizzo condivisibile.
- **`showFigure.html`**: immagine, voto medio (`#numbers.formatDecimal(votoMedio, 1, 1) + ' / 5'`), link ad azienda e serie, dettagli, l'elenco delle recensioni con i bottoni Modifica/Elimina, la form per una nuova recensione (solo per gli utenti autenticati) oppure il link "Accedi per lasciare una recensione".
- **`showSerie.html`**: personaggi (con la form admin per aggiungerne uno), tabella dei fumetti, griglia delle figure. I titoli delle sezioni contengono i conteggi: `'Personaggi (' + ${serie.personaggi.size()} + ')'`.
- **`listRecensioni.html`**: la tabella del pannello admin, con il conteggio totale nel sottotitolo (`${recensioni.size()} + ' recensioni presenti nel sito'`) e un bottone Elimina per riga.
- **`performance.html`**: un grafico a barre fatto con normali `div`, la cui larghezza è impostata con `th:style="'width: ' + ${r.tempoMs * 100 / maxTempoMs} + '%'"`, e una tabella dei risultati (§11.2).
- **`cercaFigure.html`**: la pagina che ospita React (§10).

7.6 Il CSS (`static/css/stile.css`)

> TEORIA
>
> **[02, 09] HTML e CSS.** Separazione fra struttura (HTML) e presentazione (CSS); selettori di classe; box model; **Flexbox** (`display:flex`, `justify-content`, `align-items`, `flex-wrap`); **variabili CSS**; **responsive design** con meta viewport e media query; unità relative `rem` / `em`.

Il foglio di stile è lo stesso del progetto di riferimento, ricolorato con un tema viola scuro. Usa:

- **variabili CSS** in `:root` (`--bg-primary`, `--accent-400`, `--radius-md`...), riusate con `var(--accent-400)`: il tema si cambia in un solo punto;

- il font **Calibri** (`font-family: Calibri, Carlito, 'Segoe UI', Arial, sans-serif`): l'elenco dopo Calibri sono i *fallback* usati se il font non è installato;
- **Flexbox** per la navbar, le barre di azioni e la paginazione;
- **CSS Grid** per le griglie di card: `grid-template-columns: repeat(auto-fit, minmax(220px, 1fr))` crea tante colonne quante ne entrano, e il layout diventa responsive senza media query. Grid non è nelle slide, ma è il fratello bidimensionale di Flexbox;
- **media query** per gli schermi piccoli e unità `rem` ; `object-fit: contain` per le immagini delle figure.

Le immagini, il CSS e il bundle React stanno in `static/` , i template in `templates/` 03 · sl. 76 .

8. Sicurezza (package `security`)

TEORIA

10 · sl. 2–4 *Autenticazione* = chi sei; *autorizzazione* = che cosa puoi fare, in base al ruolo. 10 · sl. 5 Costruirsela da soli "sarebbe estremamente ripetitivo e rischioso": si usa Spring Security. 10 · sl. 13–19 La classe di configurazione crea tre `@Bean`: `UserDetailsService`, `PasswordEncoder`, `SecurityFilterChain`.

Il package contiene due classi: `CustomUserDetailsService` e `SecurityConfiguration` (annotata `@Configuration` ed `@EnableWebSecurity`).

8.1 Dove stanno le credenziali: `CustomUserDetailsService`

```
@Service
public class CustomUserDetailsService implements UserDetailsService {
    @Autowired
    private CredentialsRepository credentialsRepository;

    @Override
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        Credentials credentials = credentialsRepository.findByUsername(username)
                .orElseThrow(() -> new UsernameNotFoundException("Utente non trovato: " +
username));
        return new User(credentials.getUsername(), credentials.getPassword(),
                List.of(new SimpleGrantedAuthority("ROLE_" + credentials.getRole())));
    }
}
```

La slide 10 · sl. 14 usa un `JdbcUserDetailsManager` con due query SQL scritte a mano sulla tabella `credentials`. FigureDB fa la stessa cosa **passando dal repository JPA**, come il progetto di riferimento: al login Spring Security chiama `loadUserByUsername`, che cerca le credenziali con `findByUsername` e le restituisce come oggetto `UserDetails` (qui `User` è la classe di Spring Security, non l'entità). Spring confronta poi la password digitata con quella cifrata tramite il `PasswordEncoder`.

Il prefisso `ROLE_`. Il DB contiene `ADMIN` o `DEFAULT`; l'authority diventa `ROLE_ADMIN` o `ROLE_DEFAULT`. È la convenzione di Spring Security: `hasRole('ADMIN')` cerca proprio `ROLE_ADMIN`. Per questo la configurazione e i template usano `hasRole`, e `RecensioneController.isAdmin()` confronta con la stringa `"ROLE_ADMIN"`.

8.2 `PasswordEncoder` e `DaoAuthenticationProvider`

```
@Bean
public PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }

@Bean
public DaoAuthenticationProvider authenticationProvider() {
    DaoAuthenticationProvider provider = new DaoAuthenticationProvider(userDetailsService);
    provider.setPasswordEncoder(passwordEncoder());
    return provider;
}
```

`BCryptPasswordEncoder` 10 · sl. 15: BCrypt è un hash **lento e con sale**. Lo stesso testo produce ogni volta un hash diverso (`$2a$10$…` , dove `10` è il fattore di costo), quindi le password non si ricavano con tabelle precalcolate. Gli hash delle tre utenze di prova in `data.sql` sono stati generati con lo stesso algoritmo 10 · sl. 22–23.

Il `DaoAuthenticationProvider` è il componente che esegue materialmente il login: usa il `UserDetailsService` per caricare l'utente e il `PasswordEncoder` per verificare la password. Viene registrato nella catena con `.authenticationProvider(...)` . All'avvio Spring scrive un *warning* ("Global AuthenticationManager configured with an AuthenticationProvider bean…"): avverte solo che il `UserDetailsService` non verrà usato una seconda volta in automatico. È innocuo e compare anche nel progetto di riferimento.

8.3 La catena di filtri: chi può accedere a cosa

```
.authorizeHttpRequests(auth -> auth
        .requestMatchers("/css/**", "/js/**", "/images/**", "/favicon.ico").permitAll()
        .requestMatchers(HttpMethod.GET, "/api/**").permitAll()
        .requestMatchers(HttpMethod.OPTIONS, "/api/**").permitAll()
        .requestMatchers(HttpMethod.GET, "/admin/**").hasRole(Credentials.ADMIN_ROLE)
        .requestMatchers(HttpMethod.POST, "/admin/**").hasRole(Credentials.ADMIN_ROLE)
        .requestMatchers(HttpMethod.GET, "/", "/index", "/login", "/register").permitAll()
        .requestMatchers(HttpMethod.POST, "/register").permitAll()
        .requestMatchers(HttpMethod.GET, "/figure", "/figure/**", "/serie", "/serie/**",
                "/fumetti", "/fumetti/**", "/aziende", "/aziende/**",
                "/scrittori/**", "/disegnatori/**", "/editori/**").permitAll()
        .anyRequest().authenticated())
```

> TEORIA
>
> 10 · sl. 17 "Le regole si leggono dall'alto verso il basso e **la prima che matcha vince**."

Le regole, lette in ordine:

1. Risorse statiche (CSS, JavaScript compreso il bundle React, immagini): tutti.
2. API REST in lettura (`GET /api/**`) e richieste `OPTIONS` (il *preflight* CORS del browser, §8.5): tutti.
3. `/admin/**` in GET e POST: solo chi ha `ROLE_ADMIN` . Le regole admin stanno **prima** di quelle pubbliche: `/admin/figure/new` non rischia di essere catturato da una regola più generica.
4. Home, login e registrazione: tutti.
5. Consultazione del catalogo, **solo in GET**: tutti. La `POST /figure/{id}/recensione` non rientra qui (è una POST), quindi arriva alla regola finale e richiede l'autenticazione.
6. Tutto il resto (`/profilo` , le POST delle recensioni, `/logout` …): utente autenticato.

Autorizzazione a due livelli. Spring Security dice *chi* può chiamare un URL: per esempio "un utente autenticato può fare `POST /figure/3/recensione/5/edit` ". Non può sapere se la recensione 5 è sua. Questo controllo di proprietà, **a livello di oggetto**, lo fa `RecensioneService.modifica` / `elimina` . La specifica chiede entrambi i livelli.

8.4 Login e logout

```
.formLogin(form -> form
        .loginPage("/login")
        .defaultSuccessUrl("/", true)
        .failureUrl("/login?error=true")
        .permitAll())
.logout(logout -> logout
        .logoutUrl("/logout")
        .logoutSuccessUrl("/")
        .invalidateHttpSession(true)
        .deleteCookies("JSESSIONID")
        .clearAuthentication(true)
        .permitAll());
```

Corrisponde riga per riga a 10 · sl. 18–19. Unica differenza: dopo il login si va a `/` invece che a `/success`. Con `true` il redirect alla home è forzato anche se l'utente aveva chiesto un'altra pagina protetta 10 · sl. 32. La form di login invia `username` e `password` a `POST /login`, che Spring Security intercetta da solo: non serve scrivere quel metodo nel controller.

Dopo l'autenticazione il server crea una **sessione** e manda al browser il cookie `JSESSIONID`, che il browser rimanda a ogni richiesta 10 · sl. 4; 01, "Gestione della sessione". È l'approccio *stateful* descritto in 12-3 · sl. 5. Un anonimo che tenta di entrare in una pagina protetta viene rediretto alla form di login 10 · sl. 31.

8.5 CSRF e CORS

TEORIA

10 · sl. 24–30 **Cross-Site Request Forgery.** Un sito malevolo può far inviare al browser una POST verso il nostro sito: il browser allega automaticamente il cookie di sessione e il server crede che la richiesta sia legittima. La difesa è un **token casuale** che il server inserisce nella pagina e che il client deve rimandare; il sito malevolo non può leggerlo. Con `th:action` Thymeleaf lo inserisce da solo.
12-2 · sl. 8 **CORS:** il browser impedisce a una pagina servita da un'origine (es. `localhost:5173`) di chiamare un'altra origine, a meno che il server non lo permetta configurando una `CorsConfigurationSource`.

```
.cors(cors -> cors.configurationSource(corsConfigurationSource()))
.csrf(csrf -> csrf.ignoringRequestMatchers("/api/**"))
```

- **CSRF** resta attivo per tutto il sito MVC (form Thymeleaf) ed è **disattivato solo per `/api/**`**. Le API del progetto sono tutte in sola lettura (GET), e una GET non ha bisogno del token: è la stessa configurazione del progetto di riferimento.
- **CORS** autorizza le origini `http://localhost:5173` (il server di sviluppo di Vite) e `http://localhost:3000`, solo sulle URL `/api/**`. Serve quando si sviluppa il widget con `npm run dev`: la pagina è servita da Vite su 5173 e le chiamate vanno a Spring Boot su 8081, cioè un'**altra origine**. In produzione il widget è servito da Spring Boot stesso (stessa origine) e CORS non entra in gioco. Il `vite.config.ts` ha comunque anche un *proxy* che inoltra `/api` a `localhost:8081`.

ALL'ORALE

"Fai entrare in /admin anche un ruolo EDITOR": `hasAnyRole("ADMIN", "EDITOR")` nelle due regole `/admin/**`, e la costante `EDITOR_ROLE` in `Credentials`. **"Perché non si usano `@PreAuthorize` sui metodi?"**: perché tutte le risorse admin hanno lo stesso prefisso, e una regola sull'URL è sufficiente; `@PreAuthorize` (con `@EnableMethodSecurity`) servirebbe per regole più fini. **"Se aggiungessi una POST alle API REST, che cosa dovresti cambiare?"**: togliere `/api/**` da `ignoringRequestMatchers` (o usare un token CSRF in cookie leggibile da JavaScript, `CookieCsrfTokenRepository.withHttpOnlyFalse()`, rimandato nell'header `X-XSRF-TOKEN`) e richiedere l'autenticazione per quella POST.

9. API REST (package `rest` e `dto`)

9.1 Da `@Controller` a `@RestController`

> TEORIA
>
> 11 · sl. 2–4 Un SIW può offrire i suoi dati anche ad altri client (app, frontend JavaScript): invece di pagine HTML si restituiscono dati in un formato di scambio, **JSON**. 11 · sl. 8–15 **REST**: le *risorse* sono identificate da URL (`/movies/12`), le *azioni* sono i metodi HTTP (GET legge, POST crea, PUT aggiorna, DELETE cancella), le *rappresentazioni* sono in JSON. 11 · sl. 21–22 `@RestController` : il metodo restituisce l'oggetto e il framework lo serializza in JSON. 12-2 · sl. 6 Codici di stato: GET → 200/404; POST → 201 Created/400; PUT → 200/404/400; DELETE → 204 No Content/404.

`@RestController` = `@Controller` + `@ResponseBody` su ogni metodo: il valore restituito non è il nome di una vista, ma il **corpo della risposta**, convertito in JSON da Jackson.

9.2 Gli endpoint

`FigureRestController` è annotato `@RequestMapping("/api/figure")` : il prefisso vale per tutti i metodi della classe.

Metodo e URL	Accesso	Risposta
`GET /api/figure`	pubblico	200 + tutte le figure (`FigureDTO[]`)
`GET /api/figure?nome=miku`	pubblico	200 + figure il cui nome contiene "miku"
`GET /api/figure/{id}`	pubblico	200 + `FigureDTO` , oppure **404**
`GET /api/figure/{id}/recensioni`	pubblico	200 + `RecensioneDTO[]` , oppure **404** se la figure non esiste

```
@GetMapping
public ResponseEntity<List<FigureDTO>> tutte(@RequestParam(required = false) String nome) {
    List<FigureDTO> figure = (nome == null || nome.isBlank()
            ? figureService.findAll()
            : figureService.cercaPerNome(nome))
            .stream()
            .map(FigureDTO::from)
            .toList();
    return ResponseEntity.ok(figure);
}

@GetMapping("/{id}")
public ResponseEntity<FigureDTO> una(@PathVariable Long id) {
    return figureService.findByIdWithAziendaESerie(id)
            .map(FigureDTO::from)
            .map(ResponseEntity::ok)
            .orElseGet(() -> ResponseEntity.notFound().build());
}
```

- **`ResponseEntity<T>`** permette di scegliere il codice di stato oltre al corpo: `ok(corpo)` produce 200, `notFound().build()` produce 404 senza corpo.
- **`@RequestParam(required = false) String nome`** : senza parametro si restituisce tutto il catalogo, con il parametro si delega alla `@Query` di ricerca (§4.3). La ricerca la fa il **DBMS**, non Java.

- **Lo stream** trasforma ogni entità in un DTO: `.map(FigureDTO::from)` è un *method reference*, equivalente a `.map(f -> FigureDTO.from(f))`.
- **La catena di `Optional` in `una`**: se la figure esiste, la si trasforma in DTO e poi in una risposta 200; se non esiste, `orElseGet` costruisce il 404. È la versione funzionale di `if (opt.isPresent()) … else …`.
- Il controller REST usa **gli stessi service** di quello MVC: la logica non è duplicata. È l'argomento di `08 · sl. 42`: "stessa logica usata da API, batch, test".

9.3 I DTO: perché non restituire le entità

> TEORIA
>
> `11 · sl. 23–28` **Dipendenze cicliche.** Serializzare un'entità con associazioni bidirezionali (Movie ↔ Artist) produce un ciclo infinito. Il prof propone `@JsonIgnore` oppure `@JsonIdentityInfo`. `12-2 · sl. 19` Lato React, le interfacce TypeScript replicano i DTO Java del backend.

FigureDB usa i **DTO** (*Data Transfer Object*): classi che contengono solo i campi da esporre, costruite con un metodo statico `from(...)`.

```
public class FigureDTO {
    private Long id;
    private String nome;
    private LocalDate dataUscita;
    private Float altezza;
    private String materiale;      // etichetta leggibile, es. "PVC e ABS"
    private Float prezzo;
    private String immagine;
    private String azienda;        // solo il nome, non l'oggetto Azienda
    private String serie;          // solo il nome, non l'oggetto Serie

    public static FigureDTO from(Figure f) {
        FigureDTO dto = new FigureDTO();
        dto.id = f.getId();
        ...
        dto.materiale = f.getMateriale() != null ? f.getMateriale().getLabel() : null;
        dto.azienda = f.getAzienda() != null ? f.getAzienda().getNome() : null;
        dto.serie = f.getSerie() != null ? f.getSerie().getNome() : null;
        return dto;
    }
    // getter e setter: servono a Jackson per produrre il JSON
}
```

Tre vantaggi:

1. **Niente cicli**: il DTO è "piatto" (`azienda` è una stringa, non l'oggetto `Azienda`, che a sua volta contiene la lista delle figure…). Le entità hanno comunque `@JsonIgnore` sulle collezioni come seconda protezione.
2. **Niente caricamenti a sorpresa**: serializzando un'entità, Jackson percorrerebbe le associazioni LAZY, innescando query aggiuntive. Il DTO contiene solo i campi scelti, costruiti da dati già caricati (`findAllByOrderByNomeAsc` e `cercaPerNome` caricano azienda e serie insieme alla figure).
3. **Disaccoppiamento**: si può rinominare un campo dell'entità senza rompere il contratto dell'API usato da React. Si evita anche di esporre per sbaglio dati che non devono uscire (per esempio la password, se si serializzasse `Credentials`).

`RecensioneDTO` ha `id`, `titolo`, `testo`, `voto`, `data` e `autore` come stringa (`nomeCompleto`, oppure "Anonimo"). Il terzo DTO, `FetchScenarioRisultato`, non esce dalle API: trasporta i risultati del confronto

sulle strategie di fetch verso la pagina `performance.html` (§11.2).

Esempio di risposta di `GET /api/figure?nome=grendizer`:


```
[{ "id": 951, "nome": "Bandai Ufo Robot Grendizer", "dataUscita": "2026-04-22",
   "altezza": 16.0, "materiale": "ABS", "prezzo": 80.0,
   "immagine": "/images/figure/Bandai_Ufo_Robot_Grendizer/bandai-the-robot-spirits-ufo-robot-grendizer_01.jpg",
   "azienda": "Bandai", "serie": "Ufo Robot" }]
```


ALL'ORALE

"Aggiungi un endpoint con le figure di una serie." In `FigureRestController`: `@GetMapping("/serie/{serieId}") public ResponseEntity<List<FigureDTO>> perSerie(@PathVariable Long serieId)`, che chiama `figureService.findBySerieId(serieId)` e mappa con `FigureDTO::from`. Per non avere N+1 sul nome dell'azienda, si aggiunge `@EntityGraph(attributePaths = {"azienda", "serie"})` a `findBySerieIdOrderByNomeAsc`. L'URL è già pubblico (`GET /api/**`). **"E un endpoint per la media dei voti?"** `@GetMapping("/{id}/voto-medio")` che restituisce `ResponseEntity.ok(recensioneService.votoMedio(id))`: la `@Query` con `AVG` esiste già.

10. Il frontend React (`frontend/react-figure`)

10.1 Perché React proprio qui

> TEORIA
>
> 12-1 · sl. 3–4 React è una libreria per costruire interfacce componendo **componenti**, con approccio **dichiarativo**: "dato questo stato, l'UI appare così". 12-1 · sl. 20–22 **Virtual DOM**: React aggiorna solo le parti cambiate.

La ricerca delle figure è il punto in cui l'interattività ha un vantaggio reale: mentre l'utente scrive o cambia un filtro, la tabella si aggiorna **senza ricaricare la pagina**. Con Thymeleaf ogni filtro sarebbe una nuova richiesta e una nuova pagina HTML completa. Il resto del sito resta Thymeleaf: è un'architettura **ibrida**, in cui React è "montato" dentro un `<div>` di una pagina server-side (`/figure/cerca`, link "Figure React" nella navbar).

10.2 Il progetto Vite e la build

> TEORIA
>
> 12-1 · sl. 2–8 Lo stack della lezione: **TypeScript** (JavaScript con i tipi), **Vite** (build tool che sostituisce Create React App), **Axios** per le chiamate HTTP. 12-1 · sl. 12–13 "Axios è una libreria per fare chiamate HTTP, più comoda di fetch": converte da sola il JSON e lancia un'eccezione per le risposte di errore (4xx, 5xx).

Il widget sta in una cartella separata, `frontend/react-figure/`, alla radice del progetto:

```
frontend/react-figure/
├── package.json        dipendenze: react 18, react-dom, axios; dev: vite, typescript, @vitejs/plugin-react
├── vite.config.ts      base '/js/react-figure/', outDir ../../src/main/resources/static/js/react-figure
├── index.html          pagina di prova per "npm run dev"
└── src/
    ├── main.tsx        punto di ingresso: monta il componente sul div #figure-react-root
    ├── api.ts          istanza Axios e funzione getFigure(nome)
    ├── types.ts        interfaccia TypeScript Figure (replica FigureDTO)
    └── components/FigureSearchWidget.tsx
```

Il ciclo di lavoro:

1. Si scrive il codice TypeScript/JSX in `src/`.
2. `npm install` (una volta) scarica le dipendenze in `node_modules/` (esclusa da Git).
3. `npm run build` esegue `tsc -b` (controllo dei tipi) e poi `vite build`, che traduce TypeScript e JSX in JavaScript normale e unisce tutto in **un solo file**: `src/main/resources/static/js/react-figure/assets/index.js`.
4. Spring Boot serve quel file come qualunque risorsa statica; la pagina `cercaFigure.html` lo carica.

Le opzioni di `vite.config.ts` che rendono possibile l'integrazione:

```
export default defineConfig({
  plugins: [react()],
  base: '/js/react-figure/',
  server: { port: 5173, proxy: { '/api': 'http://localhost:8081' } },
  build: {
    outDir: fileURLToPath(new URL('../../src/main/resources/static/js/react-figure',
import.meta.url)),
    emptyOutDir: true,
    rollupOptions: { output: { entryFileNames: 'assets/index.js', ... } },
  },
})
```

- **`outDir`** scrive il risultato direttamente dentro `static/`: non serve copiare file a mano.
- **`entryFileNames: 'assets/index.js'`**: di default Vite aggiunge un *hash* al nome (`index-a1b2c3.js`) per evitare la cache; qui il nome è fisso, così il template Thymeleaf può riferirlo staticamente. Contro la cache del browser il template aggiunge un parametro di versione (`index.js?v=1`).
- **`server.proxy`**: con `npm run dev` il widget gira su `localhost:5173` e le chiamate a `/api` vengono inoltrate a Spring Boot su 8081.

Il bundle è già compilato e incluso nel progetto: per l'esame basta avviare Spring Boot. La build va rifatta solo se si modifica il codice React.

10.3 Come viene montato nella pagina

```
<!-- cercaFigure.html -->
<div id="figure-react-root" th:attr="data-api-base=@{/api}"></div>
<script type="module" th:src="@{/js/react-figure/assets/index.js(v=1)}"></script>
```

```
// main.tsx
const mount = document.getElementById('figure-react-root')
if (mount) {
  createRoot(mount).render(<StrictMode><FigureSearchWidget /></StrictMode>)
}
```

```
// api.ts
function baseUrl(): string {
  const el = document.getElementById('figure-react-root')
  return el?.dataset?.apiBase ?? '/api'
}
export const api = axios.create({ baseURL: baseUrl(), timeout: 10_000 })

export async function getFigure(nome?: string): Promise<Figure[]> {
  const { data } = await api.get<Figure[]>('/figure', {
    params: nome && nome.trim() !== '' ? { nome } : undefined,
  })
  return data
}
```

- **Il `div` è il "punto di aggancio"**: Thymeleaf produce un `div` vuoto, e React ci disegna dentro. L'attributo `data-api-base` (scritto con `@{/api}`, quindi con il context path corretto) dice al widget dove sono le API; JavaScript lo legge con `el.dataset.apiBase`.
- **`axios.create`** crea un'istanza con l'URL di base e un timeout di 10 secondi. `api.get<Figure[]>('/figure', { params })` chiama `/api/figure?nome=…`: Axios costruisce la query string e converte il JSON. Il tipo generico `<Figure[]>` dice a TypeScript che forma ha la risposta.

- **`async` / `await`** 12-1 · sl. 10: `getFigure` restituisce una `Promise`; chi la chiama può aspettare il risultato con `await` come se fosse codice sequenziale.

```ts
// types.ts: replica i campi di FigureDTO
export interface Figure {
  id: number; nome: string; dataUscita: string; altezza: number; materiale: string;
  prezzo: number; immagine: string | null; azienda: string; serie: string
}
```

TEORIA

12-1 · sl. 8 **Interfacce TypeScript.** Un' `interface` descrive la forma di un oggetto; l'editor e il compilatore segnalano un campo sbagliato prima di eseguire il codice. "Le interfacce TypeScript replicano esattamente i DTO Java del backend" 12-2 · sl. 19. `LocalDate` arriva in JSON come stringa (`"2026-03-14"`), per questo `dataUscita` è `string`.

10.4 Il componente `FigureSearchWidget`

TEORIA

12-1 · sl. 15–19 Un componente è "una funzione che restituisce JSX"; il nome è in PascalCase; nelle liste la prop **`key`** è obbligatoria. JSX non è HTML: `className`, `onChange` in camelCase, espressioni tra `{ }`; rendering condizionale con `&&`. 12-1 · sl. 24–27 **`useState`** restituisce `[valore, setter]`; un cambiamento dello stato causa un nuovo render. 12-1 · sl. 28–29 **`useEffect`** gestisce i *side effect* (chiamate API, timer); l'array delle dipendenze decide quando rieseguirlo: assente = a ogni render, `[]` = solo al montaggio, `[x]` = quando cambia `x`. 12-1 · sl. 32 `useMemo` memorizza un valore calcolato.

Lo stato del componente:

```ts
const [figure, setFigure] = useState<Figure[]>([])          // risultati dall'API
const [loading, setLoading] = useState<boolean>(true)
const [errore, setErrore] = useState<string | null>(null)
const [nome, setNome] = useState<string>('')                 // testo della ricerca
const [serie, setSerie] = useState<string>('')               // filtri ('' = tutti)
const [azienda, setAzienda] = useState<string>('')
const [materiale, setMateriale] = useState<string>('')
```

1) La ricerca per nome, lato server, con debounce:

```ts
useEffect(() => {
  let annullato = false
  const timer = setTimeout(async () => {
    try {
      setLoading(true)
      const dati = await getFigure(nome)
      if (!annullato) { setFigure(dati); setErrore(null) }
    } catch (e) {
      if (!annullato) setErrore('Impossibile caricare le figure.')
    } finally {
      if (!annullato) setLoading(false)
    }
  }, 300)
  return () => { annullato = true; clearTimeout(timer) }       // cleanup
}, [nome])
```

- **Dipendenze `[nome]`**: l'effetto riparte ogni volta che cambia il testo, e anche al primo render (quindi all'apertura carica tutte le figure).
- **Debounce**: a ogni lettera digitata React esegue prima la funzione di **cleanup** dell'effetto precedente (`clearTimeout`, che annulla la ricerca programmata) e poi programma una nuova ricerca fra 300 ms. La richiesta parte solo quando l'utente smette di scrivere: invece di una richiesta per tasto ne parte una sola.
- **Il flag `annullato`**: se arriva la risposta di una ricerca vecchia dopo che l'utente ha già cambiato testo, il cleanup ha messo `annullato = true` e la risposta in ritardo non sovrascrive lo stato. Si evitano così le *race condition*.
- **`async` dentro `setTimeout`**: la callback di `useEffect` non può essere `async` direttamente 12-1 · sl. 29, quindi la funzione asincrona sta dentro.
- **`try/catch/finally`**: con Axios un errore HTTP (500, rete assente) arriva nel `catch`; `finally` spegne l'indicatore di caricamento in ogni caso.

2) I filtri per serie, azienda e materiale, lato client:

```
function valoriDistinti(figure: Figure[], campo: 'serie' | 'azienda' | 'materiale'): string[]
{
  return Array.from(new Set(figure.map(f => f[campo]).filter(Boolean))).sort()
}

const serieDisponibili = useMemo(() => valoriDistinti(figure, 'serie'), [figure])

const figureFiltrate = useMemo(() => {
  return figure.filter(f =>
    (!serie || f.serie === serie) &&
    (!azienda || f.azienda === azienda) &&
    (!materiale || f.materiale === materiale))
}, [figure, serie, azienda, materiale])
```

- Le **opzioni delle tendine** si ricavano dai risultati: `map` estrae il campo, `filter(Boolean)` scarta i valori vuoti, `new Set` elimina i duplicati, `Array.from(...).sort()` li riporta in un array ordinato. Il tipo `'serie' | 'azienda' | 'materiale'` (unione di stringhe letterali) impedisce di passare un nome di campo sbagliato.
- **Il filtro** combina le tre condizioni in `&&`; una tendina vuota (`''`) significa "nessun filtro", per questo ogni condizione inizia con `!serie ||`.
- **`useMemo`** ricalcola i risultati filtrati solo quando cambiano le figure o un filtro, non a ogni render.
- Il nome lo filtra il **server** (potrebbe esserci un catalogo enorme), mentre gli altri filtri lavorano sui risultati già scaricati, senza nuove richieste. È la stessa divisione del widget del progetto di riferimento (titolo lato server, genere lato client).

3) Il JSX:

```
<input type="text" value={nome} onChange={e => setNome(e.target.value)} placeholder="Cerca
per nome..." />
<select value={serie} onChange={e => setSerie(e.target.value)}>
  <option value="">-- Tutte le serie --</option>
  {serieDisponibili.map(s => <option key={s} value={s}>{s}</option>)}
</select>
...
{loading && <p>Caricamento...</p>}
{!loading && figureFiltrate.length === 0 && <div className="empty-state">...</div>}
{!loading && figureFiltrate.length > 0 && (
  <table className="data-table"> ...
    {figureFiltrate.map(f => (
      <tr key={f.id}> <td>{f.nome}</td> ... <td>{f.prezzo.toFixed(2)} EUR</td>
        <td><a href={`/figure/${f.id}`} className="btn-small">Dettagli</a></td>
      </tr>))}
  </table>)}
```

- **Componenti controllati** 12-2 · sl. 13–14: ogni input ha `value={stato}` e un `onChange` che aggiorna lo stato. Il valore vive nello stato React, non nel DOM.
- **Rendering condizionale con `&&`**: mostra "Caricamento...", il messaggio "nessun risultato" oppure la tabella. Gli stati di caricamento ed errore sono richiesti dalla specifica.
- `key={f.id}` permette a React di riconoscere le righe fra un render e l'altro.
- Il link "Dettagli" porta alla pagina Thymeleaf `/figure/{id}`: React e Thymeleaf convivono.
- **Classi CSS condivise**: il widget usa `data-table`, `empty-state`, `btn-small` di `stile.css`, quindi ha lo stesso aspetto del resto del sito.

ALL'ORALE

"Aggiungi un filtro per prezzo massimo." Uno stato `const [prezzoMax, setPrezzoMax] = useState<string>('')`, un `<input type="number" value={prezzoMax} onChange={e => setPrezzoMax(e.target.value)} />`, e nella `filter` la condizione `(!prezzoMax || f.prezzo <= Number(prezzoMax))`, aggiungendo `prezzoMax` alle dipendenze di `useMemo`. Poi `npm run build`. **"Perché qui non serve CORS?"** In produzione pagina e API hanno la stessa origine (`localhost:8081`); CORS serve solo con il server di sviluppo di Vite su 5173 (§8.5). **"Perché l'array di dipendenze `[nome]` e non `[]`?"** Con `[]` la ricerca partirebbe solo all'apertura e non a ogni modifica del testo.

11. Dati iniziali e analisi delle prestazioni

11.1 `data.sql`

> TEORIA
>
> 03 · sl. 78–79 Il modo base per popolare il DB è un file SQL nella stessa cartella di `application.properties`, eseguito all'avvio. 10 · sl. 22–23 Si possono inserire anche i dati di un utente ADMIN: "la password deve essere cifrata con lo stesso metodo usato dal codice".

`data.sql` contiene 3 utenti con credenziali, 17 serie con 70 personaggi, 9 aziende, 20 figure, 6 editori, 7 scrittori, 8 disegnatori, 14 fumetti con i loro collegamenti ai disegnatori e 4 recensioni di esempio. Le utenze di prova sono `admin` / `admin` (ruolo ADMIN), `mario` / `user` e `giulia` / `user` (ruolo DEFAULT).

Il problema da risolvere: con `spring.sql.init.mode=always` lo script gira a **ogni** avvio, e con `ddl-auto=update` le tabelle e i dati restano. Un `INSERT` normale, al secondo avvio, duplicherebbe i dati o violerebbe un vincolo `UNIQUE`. Per questo ogni INSERT è **idempotente**:

```
INSERT INTO figure (id, nome, data_uscita, altezza, materiale, immagine, prezzo, descrizione,
azienda_id, serie_id)
SELECT nextval('figure_seq'), 'S.H. Figuarts Spider-Man', DATE '2026-03-14', 15.5, 'PVC',
       '/images/figure/SHFiguarts_Spider_Man_Snodabile/item_0000015491_dhdQz2FY_01_q.jpg',
49.0,
       'Spider-Man snodabile con ...', a.id, s.id
FROM azienda a, serie s WHERE a.nome = 'S.H. Figuarts' AND s.nome = 'Spider-Man'
AND NOT EXISTS (SELECT 1 FROM figure WHERE nome = 'S.H. Figuarts Spider-Man'
                AND azienda_id = a.id AND data_uscita = DATE '2026-03-14');
```

Come leggerlo:

- **`INSERT … SELECT`** al posto di `INSERT … VALUES`: la riga da inserire è il risultato di una SELECT. Se la SELECT non restituisce righe, non si inserisce nulla.
- **`AND NOT EXISTS (…)`** è la condizione di idempotenza: se la figure esiste già (stesso nome, azienda e data, cioè l'identificatore naturale), la SELECT è vuota e l'INSERT non fa nulla.
- **Le FK si risolvono per nome**: `FROM azienda a, serie s WHERE a.nome = … AND s.nome = …` trova gli id di azienda e serie, senza scrivere a mano numeri che cambierebbero da un database all'altro.
- **`nextval('figure_seq')`** prende l'id dalla stessa sequenza che usa Hibernate. Con `GenerationType.AUTO`, Hibernate 6+ crea una sequenza `<tabella>_seq` per ogni entità, con incremento 50: Hibernate riserva blocchi di 50 id alla volta, e i valori presi dallo script non si sovrappongono a quelli che userà dopo.
- **Le password** sono già cifrate con BCrypt (`$2a$10$…`), generate con lo stesso `BCryptPasswordEncoder` dell'applicazione.
- **L'ordine conta**: prima utenti, serie, aziende, editori e autori, poi figure e fumetti (che li referenziano), poi la tabella di join `fumetto_disegnatore` e le recensioni.

Rispetto al progetto originale (che leggeva file CSV con un seeder Java), `data.sql` è più semplice da spiegare ed è lo stesso meccanismo del progetto di riferimento. Lo svantaggio è che lo script scrive direttamente nel DB, senza passare dalla validazione delle entità.

11.2 `FetchBenchmarkService` e la pagina `/admin/performance`

È l'analisi sperimentale richiesta dalla specifica (§8.2), resa disponibile nell'interfaccia web dell'amministratore (card "Analisi Prestazioni" nella dashboard). Lo **stesso caso d'uso**, "carica tutte le figure con la relativa azienda", viene eseguito con tre strategie, e per ciascuna si misurano il numero di query SQL, il tempo e gli oggetti caricati.

> TEORIA
>
> 06 · sl. 21–22 **Lazy vs Eager.** 06 · sl. 33–34 **Il problema 1+N:** "l'ORM ha caricato i dati di tutti i Customer, quindi ha fatto una SELECT per caricare l'address di ogni customer". 06 · sl. 35–37 **Soluzioni:** `JOIN FETCH` nel repository oppure `@EntityGraph`; "una sola query, tempi 1/10". 06 · sl. 42–46 Il lazy loading è realizzato con il **pattern Proxy**: al posto dell'oggetto collegato c'è un sostituto che esegue la query alla prima chiamata di un metodo.

```
@Transactional(readOnly = true)
public List<FetchScenarioRisultato> eseguiConfronto() {
    Statistics statistics = entityManager.getEntityManagerFactory()
            .unwrap(SessionFactory.class).getStatistics();
    risultati.add(eseguiScenario(statistics, "Default (LAZY)", "...",
            () -> conta(figureRepository.findAll())));
    risultati.add(eseguiScenario(statistics, "JOIN FETCH esplicito", "...",
            () -> conta(figureRepository.findAllWithAzienda())));
    risultati.add(eseguiScenario(statistics, "@EntityGraph", "...",
            () -> conta(figureRepository.findAllBy())));
    ...
}

private int[] conta(List<Figure> figure) {
    for (Figure f : figure) {
        f.getAzienda().getNome();        // simula la vista: tocca l'azienda di ogni figure
        ...
    }
}

private FetchScenarioRisultato eseguiScenario(Statistics statistics, ..., Supplier<int[]>
scenario) {
    entityManager.clear();               // svuota la cache di primo livello
    statistics.clear();                  // azzera i contatori
    StopWatch stopWatch = new StopWatch(); stopWatch.start();
    int[] conteggi = scenario.get();
    stopWatch.stop();
    long numeroQuery = statistics.getPrepareStatementCount();
    return new FetchScenarioRisultato(titolo, descrizione, numeroQuery,
stopWatch.getTotalTimeMillis(), ...);
}
```

Con i dati di esempio (20 figure di 9 aziende) la pagina mostra:

Strategia	Metodo del repository	Query SQL
Default (LAZY)	`findAll()` ereditato	**10** = 1 per le figure + 9, una per ogni azienda distinta
JOIN FETCH esplicito	`findAllWithAzienda()` con `@Query`	**1**
@EntityGraph	`findAllBy()` con `@EntityGraph(attributePaths = "azienda")`	**1**, stessa SQL della precedente

- **Strategia LAZY**: `findAll()` carica solo le figure; ogni `azienda` è un **proxy**, e `f.getAzienda().getNome()` innesca una SELECT. Le query aggiuntive sono 9 e non 20, perché le aziende già caricate si riusano grazie alla cache del *persistence context*: N è il numero di aziende *distinte*, non di figure.
- **`entityManager.clear()`** prima di ogni misura: il persistence context è una cache di primo livello. Senza svuotarlo, la seconda strategia troverebbe in memoria le aziende caricate dalla prima e il confronto sarebbe falsato.
- **`unwrap(SessionFactory.class)`** passa dall'API standard JPA a quella specifica di Hibernate, che offre `Statistics`; il contatore funziona grazie a `generate_statistics=true` in `application.properties`. `getPrepareStatementCount()` conta le istruzioni SQL preparate, cioè le query.
- **`Supplier<int[]>`** è un'interfaccia funzionale di Java: ogni strategia si passa come lambda (`() -> conta(figureRepository.findAll())`), e `eseguiScenario` la esegue misurandola.
- **`@PersistenceContext EntityManager`**: l'oggetto JPA "di base" su cui si appoggiano i repository; qui serve per svuotare la cache e arrivare alle statistiche.
- Il metodo è `readOnly`: l'analisi non scrive nulla. I tempi in millisecondi variano a ogni esecuzione (bottone "Riesegui il confronto"); il numero di query invece è sempre lo stesso.

ALL'ORALE

"Che cosa succede al crescere dei dati?" (domanda prevista dalla specifica). Con LAZY il numero di query cresce **linearmente** con il numero di aziende distinte (1 + N), e ogni query costa un round-trip di rete verso il DB. Con JOIN FETCH ed EntityGraph resta **1** query: cresce la dimensione del risultato, non il numero di round-trip. Il fetch join di **collezioni**, invece, moltiplica le righe (prodotto cartesiano) e impedisce la paginazione nel DB: lì conviene `@BatchSize` (carica N collezioni con una `IN (…)`) oppure query separate. **"Come lo faresti vedere dalla console?"** Con `show-sql=true` ogni SELECT compare nel log: aprendo `/admin/performance` si vedono le 10 SELECT della prima strategia e la singola SELECT con `left join azienda` delle altre due.

12. Test automatici

`src/test/java` contiene una sola classe, come il progetto di riferimento:

```
@SpringBootTest
class FiguredbApplicationTests {
    @Test
    void contextLoads() { }
}
```

`@SpringBootTest` avvia l'intero contesto Spring: crea tutti i bean, si collega al DB, fa creare le tabelle a Hibernate ed esegue `data.sql`. Il test non contiene asserzioni, ma fallisce se qualcosa di questo processo non funziona: un mapping JPA sbagliato, una `@Query` JPQL non valida (le query vengono validate all'avvio), un nome di metodo derivato che non corrisponde a un attributo, un errore nello script SQL. È un "test di fumo" economico.

Il test usa la configurazione principale, quindi richiede PostgreSQL attivo con il database `figuredb_sempl` (§14).

13. Due casi d'uso seguiti dall'inizio alla fine

13.1 L'amministratore inserisce una nuova figure

1. **Browser**: `GET /admin/figure/new`. **Spring Security** controlla la regola `GET /admin/** → hasRole(ADMIN)`. Un anonimo viene rediretto a `/login`; un utente DEFAULT riceve 403.
2. **`FigureController.createForm`**: mette nel model una `Figure` vuota come `figureForm` e, con `aggiungiDatiForm`, le liste di aziende, serie e materiali. Vista: `formFigure`.
3. **Thymeleaf** genera la form; poiché `figureForm.id` è nullo, il titolo è "Nuova Figure" e l'azione è `/admin/figure`. `th:action` aggiunge il campo nascosto `_csrf`.
4. **Browser**: `POST /admin/figure` con `nome=…&aziendaId=3&serieId=5&dataUscita=2026-03-14&altezza=15&materiale=PVC&prezzo=49&_csrf=…`.
5. **Spring Security** verifica il token CSRF e il ruolo (regola `POST /admin/**`).
6. **Binding**: `StringTrimmerEditor` toglie gli spazi; Spring converte `dataUscita` in `LocalDate`, `altezza` e `prezzo` in `Float`, `materiale` nella costante dell'enum; `@Valid` applica `@NotBlank`, `@Positive`… Errori di conversione o di validazione finiscono in `BindingResult`, e in quel caso si torna alla form con i messaggi di `message.properties`.
7. **`FigureService.inserisciACatalogo`**: il proxy transazionale apre una transazione **SERIALIZABLE**; `findById` dell'azienda (SELECT), `findById` della serie (SELECT), `existsByNomeIgnoreCaseAndAziendaIdAndDataUscita` (SELECT), poi `save` (INSERT al commit) e una riga di log `info`.
8. **Esito**: con un duplicato, `OperazioneNonValidaException` provoca il rollback e il controller ripresenta la form con il messaggio in alto. Con successo: commit e `redirect:/figure/{id}` (302).
9. **Browser**: `GET /figure/{id}`, cioè il pattern Post/Redirect/Get; `FigureController.show` mostra la scheda della nuova figure.

13.2 Un utente recensisce una figure dal sito

1. `GET /figure/7`: `GlobalController` mette nel model `userDetails` e `currentUserId`; `FigureController.show` aggiunge `figure` (con azienda e serie da `@EntityGraph`), `recensioni` (con autore) e `votoMedio` (`AVG`).
2. Il template mostra la form di recensione perché `sec:authorize="isAuthenticated()"` è vero; sulle recensioni dell'utente compare il bottone "Modifica" (`recensione.autore.id == currentUserId`).
3. `POST /figure/7/recensione` con `titolo`, `testo`, `voto`: Spring Security richiede l'autenticazione (regola finale `anyRequest().authenticated()`) e il token CSRF.
4. `RecensioneController.addRecensione` ricava l'utente da `SecurityContextHolder` (`utenteCorrente()`) e chiama il service.
5. `RecensioneService.inserisci`: figure → `existsByFigureIdAndAutoreId` → `save`. Prima dell'INSERT, `@PrePersist` imposta la data di oggi. L'ultima difesa è il vincolo `uk_recensione_figure_autore` nel DB.
6. Redirect al dettaglio. Se l'utente aveva già recensito la figure, il messaggio di `RecensioneDuplicataException` arriva alla pagina come flash attribute `errore`.
7. L'amministratore vede la nuova recensione anche in `/admin/recensioni`, in cima alla tabella (ordinata per data decrescente), e può cancellarla da lì.

14. Punti deboli e differenze dalla specifica

Sono elencati in ordine di importanza. I primi riguardano la **specifica** e le **query nascoste**: conviene sistemarli prima della consegna, oppure prepararsi a spiegarli.

ATTENZIONE

1. La specifica in `docs/` descrive il progetto originale, non questa versione. `docs/figuredb-specifica-progetto.pdf` va aggiornata (o va spiegato perché è diversa) su questi punti:

- **Entità**: la specifica elenca `Gacha` e `VoceCollezione` (13 entità e 3 enumerazioni); qui le entità sono 11 e le enumerazioni 2. `Gacha` è stata tolta di proposito; la collezione personale non esisteva nemmeno nell'originale. Anche `Figure.edizioneLimitata` non c'è più.
- **Persistence Layer**: la specifica chiede "un fragment di repository basato su Criteria API per la ricerca dinamica del catalogo"; questa versione non lo ha (la ricerca è una `@Query` sul nome, gli altri filtri sono in React).
- **Fetch**: la tabella §8.1 dice EAGER per Personaggio→Serie, Recensione→Autore e Gacha→Azienda, e `open-in-view=false`; qui tutti i `@ManyToOne` sono LAZY e `open-in-view=true`.
- **Analisi delle prestazioni**: la specifica parla di uno script con il profilo `fetch-demo`, sulle figure di una serie, con `@NamedEntityGraph` e "120 figure, 15 per serie"; qui c'è la pagina web `/admin/performance`, su tutte le figure (20, di 9 aziende), con `@EntityGraph(attributePaths = …)`.
- **Frontend**: la specifica descrive il catalogo React su `/catalogo` con ricerca su nome/serie/azienda, filtri di prezzo ed edizioni limitate, ordinamento e pannello delle recensioni; qui il widget è su `/figure/cerca`, cerca solo per nome e filtra per serie, azienda e materiale.
- **API REST**: la specifica elenca dieci endpoint, comprese POST/PUT/DELETE sulle recensioni e le statistiche; qui ci sono quattro GET su `/api/figure`.
- **Sicurezza e transazioni**: la specifica chiede "CSRF attiva anche per le chiamate del frontend React" (qui CSRF è disattivato su `/api/**`, che però contiene solo GET) e SERIALIZABLE anche per recensione, fumetto e registrazione (qui solo per `inserisciACatalogo`, vedi punto 4).
- **Bonus**: la ricerca dei fumetti per titolo e delle serie per nome è stata tolta; la paginazione dell'elenco figure, prevista come estensione, c'è.

ATTENZIONE

2. I conteggi con `.size()` producono query N+1 durante il rendering. Con `open-in-view=true` e senza `@BatchSize`, ogni `.size()` su una collezione LAZY non ancora caricata esegue una SELECT. Misurato con i dati di esempio: `/serie` fa **1** query per le 17 serie + **17** per le figure + **17** per i fumetti; `/aziende` 1 + 9; `/admin/scrittori` 1 + 7. Con pochi dati non si nota, ma è esattamente il problema delle slide `06 · sl. 33–34`. *Rimedio in una riga*: `spring.jpa.properties.hibernate.default_batch_fetch_size=25` in `application.properties`, oppure `@BatchSize(size = 25)` (Hibernate) sopra ogni collezione. Quando il template inizializza la prima collezione, Hibernate carica insieme anche quelle delle altre entità della pagina con una sola query `… where serie_id in (?, ?, …)`: `/serie` passerebbe da 35 a 3 query. È la soluzione adottata nel progetto originale.

ATTENZIONE

3. Il 403 di Spring Security mostra la Whitelabel Error Page. Un utente DEFAULT che apre `/admin/dashboard` viene bloccato dalla catena di filtri, prima di qualsiasi controller: `GlobalExceptionHandler` non lo vede e Spring Boot non trova un template per il codice 403. Inoltre un **anonimo** che apre un URL inesistente viene rediretto al login invece di vedere il 404, perché la regola finale `anyRequest().authenticated()` scatta prima del controllo dell'URL. *Rimedio*: creare `templates/error/403.html` (Spring Boot lo usa da solo per il codice 403), oppure `.exceptionHandling(h -> h.accessDeniedPage("/accesso-negato"))` con un metodo che restituisce `error/errore`.

ATTENZIONE

4. Controlli "leggi-poi-scrivi" non SERIALIZABLE. `RecensioneService.inserisci` (una recensione per utente), `FumettoService.inserisci` / `aggiorna` (ISBN), la registrazione (username ed email) e `save` di serie e aziende (nome) controllano un duplicato e poi scrivono, ma con `@Transactional` semplice (Read Committed). Per la linea guida `07 · sl. 49`, adottata dalla specifica, dovrebbero essere SERIALIZABLE. Il danno è limitato perché c'è il vincolo `UNIQUE` sullo schema, ma in caso di concorrenza l'utente vedrebbe la pagina 500 (`DataIntegrityViolationException`) invece del messaggio. *Rimedio*: `@Transactional(isolation = Isolation.SERIALIZABLE)` su quei metodi e un `@ExceptionHandler(DataIntegrityViolationException.class)` con `@ResponseStatus(HttpStatus.CONFLICT)` in `GlobalExceptionHandler`.

ATTENZIONE

5. La modifica di una figure non controlla il vincolo di consistenza. `FigureService.aggiorna` non verifica che nome, azienda e data di uscita non coincidano con un'altra figure. Se l'admin crea un duplicato modificando una figure esistente, la violazione arriva al vincolo `uk_figure_nome_azienda_data` del DB e l'utente vede la pagina 500. *Rimedio*: una query derivata `existsByNomeIgnoreCaseAndAziendaIdAndDataUscitaAndIdNot(nome, aziendaId, data, id)` , dove `IdNot` esclude la figure stessa, e la stessa `OperazioneNonValidaException` dell'inserimento (il controller di modifica dovrebbe catturarla come fa `newFigure`).

ATTENZIONE

6. Le recensioni non passano da `@Valid` . `RecensioneController` riceve `titolo` , `testo` e `voto` come `@RequestParam` e li passa al service senza Bean Validation. I limiti sono imposti dall'HTML (`required` , `min="1" max="5"` , `maxlength="1000"`), che un utente può aggirare. Un voto 9 o un testo di 2000 caratteri arriverebbero a Hibernate, che valida l'entità prima dell'INSERT e lancia `ConstraintViolationException` : pagina 500 invece di un messaggio. *Rimedio*: controllare `voto` e lunghezze nel service lanciando `OperazioneNonValidaException` , oppure usare `@Valid @ModelAttribute Recensione` con `BindingResult` come nelle altre form.

ATTENZIONE

7. Dopo un errore di validazione la form della figure perde azienda e serie scelte. Le tendine preselezionano con `th:selected="${figureForm.azienda != null and …}"` , ma l'oggetto che arriva dalla POST non ha `azienda` né `serie` (sono `@RequestParam` separati): se un altro campo è sbagliato, la form torna con le due tendine azzerate. Lo stesso vale per le tendine di `formFumetto` . *Rimedio*: rimettere `aziendaId` e `serieId` nel model e usarli in `th:selected` , oppure impostare `figureForm.setAzienda(aziendaService.findById(aziendaId)...)` prima di tornare alla form.

ATTENZIONE

8. Controlli di cancellazione poco efficienti. `SerieService.delete`, `AziendaService.delete` e i `delete` di scrittori, disegnatori ed editori caricano **intere collezioni** (`getFumetti()`, `getFigure()`) solo per sapere se sono vuote. Sarebbe meglio una query `exists` nel DB, per esempio `figureRepository.existsBySerieId(id)` 06 · sl. 12. È un buon esempio se all'orale chiedono "trova un'inefficienza".

ATTENZIONE

9. Una query in più per ogni richiesta. `GlobalController.getCurrentUserId()` esegue `findByUsername` a **ogni** richiesta MVC di un utente loggato, anche quando la pagina non usa `currentUserId` (serve solo a `showFigure`). Si potrebbe calcolarlo solo in `FigureController.show`.

ATTENZIONE

10. Dettagli minori.

- Scrittori, disegnatori ed editori non hanno controllo dei duplicati: si possono inserire due volte gli stessi dati.
- `equals` / `hashCode` sull'identificatore naturale cambiano valore se cambia il nome: un oggetto già inserito in un `HashSet` non verrebbe più ritrovato. Il progetto usa solo `List`, quindi il problema non si presenta. `Personaggio.hashCode` usa anche `serie`, che è un proxy LAZY: calcolarlo può scatenare una query.
- `show-sql=true` più `logging.level.org.hibernate.SQL=debug` stampano ogni query **due volte** nella console.
- Il test `contextLoads` usa la configurazione principale: senza PostgreSQL attivo e senza il database `figuredb_sempl`, `mvn test` (e quindi `mvn package`) fallisce.

15. Domande probabili all'orale

Modello e JPA

- *Chi è il lato proprietario della relazione Fumetto–Disegnatore e che cosa succede togliendo* `mappedBy`*?* `Fumetto` (ha `@JoinTable`). Senza `mappedBy` JPA crea una seconda tabella di join `disegnatore_fumetti` e i due lati non sono più sincronizzati 04 · sl. 83.
- *Su quali attributi è definito* `equals`*?* Sull'identificatore naturale (nome, ISBN, email…), che è anche `unique` nello schema 04 · sl. 24. Funziona anche su oggetti non ancora salvati, che hanno id nullo.
- *Differenza tra* `@Column(nullable=false)` *e* `@NotNull`*?* Il primo è un vincolo dello schema (DB), il secondo è Bean Validation (applicazione, prima del salvataggio).
- *Perché* `EnumType.STRING`*?* Con `ORDINAL`, riordinare o inserire valori nell'enum corrompe i dati già salvati.
- *Che cosa fa* `orphanRemoval`*?* Cancella dal DB un elemento rimosso dalla collezione; da qui il rischio, se si salva l'oggetto della form con la lista vuota (§5.5).
- *A che cosa serve* `@PrePersist` *in* `Recensione`*?* È un callback del ciclo di vita: imposta la data subito prima dell'INSERT.
- *Perché* `@Valid` *sul campo* `user` *di* `Credentials`*?* Per validare a cascata l'oggetto annidato con un solo `@Valid` nel controller.

Fetch e prestazioni

- *Che cos'è il problema N+1 e dove lo mostri?* §11.2: `/admin/performance`, 10 query con LAZY contro 1 con JOIN FETCH o EntityGraph.
- *Differenza fra* `JOIN` *e* `JOIN FETCH`*?* Il primo serve solo a filtrare; il secondo inizializza anche l'associazione 06 · sl. 35.
- *Differenza fra* `JOIN FETCH` *ed* `@EntityGraph`*?* Stesso SQL; il primo si scrive in una `@Query`, il secondo si aggiunge a un metodo derivato (§4.4).
- *Che cos'è una* `LazyInitializationException` *e perché qui non capita?* È l'accesso a un proxy LAZY dopo la chiusura della sessione. Qui `open-in-view=true` tiene aperta la sessione fino alla fine del rendering.
- *Quante query fa la pagina* `/serie`*?* 1 + 17 + 17 (punto 2 del §14). Con `default_batch_fetch_size=25` sarebbero 3.
- *Come funziona la paginazione di* `/figure`*?* `PageRequest.of(pagina, 9, Sort.by("nome"))` → `LIMIT` / `OFFSET` nel DB + una `count` per il totale delle pagine (§4.4).

Transazioni

- *Perché SERIALIZABLE in* `inserisciACatalogo`*?* Leggi-poi-scrivi 07 · sl. 49: con Read Committed, due POST ravvicinate superano entrambe il controllo del duplicato.
- *Che cosa fa* `readOnly = true`*?* Niente dirty checking né flush; è un'indicazione di ottimizzazione per Hibernate e per il driver.
- *Che cosa succede se un'eccezione esce a metà di* `inserisciACatalogo`*?* Rollback completo (atomicità); le eccezioni di dominio sono `RuntimeException`.
- *Che cos'è la propagation* `REQUIRED`*?* Un metodo transazionale chiamato da un altro si unisce alla transazione esistente 07 · sl. 59.

Web, errori, sicurezza

- *Perché il redirect dopo una POST?* Pattern PRG 03-2 · sl. 13: il refresh non reinvia la POST.

- *Che differenza c'è fra i due 404 gestiti da* `GlobalExceptionHandler`*?* `RisorsaNonTrovataException`: l'URL esiste ma l'id no; `NoResourceFoundException`: l'URL non è mappato `08 · sl. 61–66`.
- *Dove si controlla che una recensione sia dell'utente?* Nel service (`modifica`, `elimina`), non nella `SecurityConfiguration`, che conosce solo gli URL.
- *Perché* `hasRole('ADMIN')` *funziona se nel DB c'è scritto* `ADMIN`*?* Perché `CustomUserDetailsService` aggiunge il prefisso `ROLE_` (§8.1).
- *Perché il logout è una form?* Con CSRF attivo `/logout` accetta solo POST con token `10 · sl. 19`.
- *Perché le API non hanno CSRF?* Sono solo GET, e una GET non modifica dati; se si aggiungesse una POST bisognerebbe riattivarlo (§8.5).
- *Perché DTO e non entità nelle API?* Cicli, lazy loading, disaccoppiamento (§9.3).

React

- *Che cosa fa l'array di dipendenze di* `useEffect`*, e a che cosa serve la funzione restituita?* §10.4: quando eseguire l'effetto; cleanup (debounce e annullamento delle risposte vecchie).
- *Perché la ricerca per nome va al server e il filtro per serie no?* Il nome può filtrare un catalogo enorme (lo fa il DB); serie, azienda e materiale lavorano sui risultati già scaricati, senza nuove richieste.
- *Che cosa fa* `npm run build` *e dove finisce il risultato?* Controllo dei tipi + bundle in `static/js/react-figure/assets/index.js`, servito da Spring Boot (§10.2).
- *Quando serve CORS?* Solo con il server di sviluppo di Vite su un'altra porta (5173).
- *Perché Axios invece di* `fetch`*?* Converte il JSON da solo e lancia un'eccezione per gli errori HTTP; con `fetch` bisognerebbe controllare `response.ok` `12-1 · sl. 12–13`.

16. Mappa rapida: concetto delle slide → dove si trova nel codice

Concetto	Slide	Dove in FigureDB
Classe Application, `main`	03 · 27	`FiguredbApplication`
`application.properties`, `ddl-auto`	03 · 23–25, 77	`application.properties` (`update`)
Popolamento del DB con SQL	03 · 78–79; 10 · 22–23	`data.sql` (INSERT idempotenti, password BCrypt)
`@Entity`, `@Id`, `@GeneratedValue`	03 · 34; 04 · 23	tutte le entità
`@Column(unique, length)`	04 · 23	`Azienda.nome`, `Fumetto.isbn`, descrizioni
Vincolo di unicità su più colonne	04 · 26, 52–54	`Figure`, `Recensione` (`@UniqueConstraint`)
equals/hashCode, identificatore naturale	04 · 20, 24	tutte le entità
`@ManyToOne` / `@OneToMany(mappedBy)`	04 · 67–76	Figure↔Azienda, Serie↔Personaggio…
`@ManyToMany` + join table	04 · 83–86	`Fumetto.disegnatori` / `fumetto_disegnatore`
`@OneToOne`	04 · 46–47; 10 · 11	`Credentials.user`
`@Table(name="users")`	10 · 10	`User`
`@Enumerated(STRING)`	07 · 4	`Serie.tipo`, `Figure.materiale`
Ereditarietà JPA	04-2	non usata (alternativa per Scrittore/Disegnatore)
JPQL, parametri `:nome`	05 · 2–14	`cercaPerNome`, `findAllWithAzienda`
Aggregazioni JPQL (`avg`)	05 · 16–19	`votoMedioPerFigure`
Repository, `save`	06 · 7–10	tutti i repository (`JpaRepository`)
Query derivate dal nome	06 · 12–15	`existsByNomeIgnoreCase`, `findByUsername` …
`@Query` + `@Param`	06 · 16–17	`FigureRepository`, `RecensioneRepository`
Fetch LAZY/EAGER, default	06 · 21–24	tabella §3.3 (tutto LAZY)
Problema 1+N, `JOIN FETCH`, `@EntityGraph`	06 · 33–37	`FetchBenchmarkService`, `/admin/performance`
Pattern Proxy / lazy load	06 · 42–46	proxy Hibernate su `azienda`, `serie`
Cascade solo sulle composizioni	06 · 49–57	`Serie.personaggi`, `Figure.recensioni`, `Credentials.user`
ACID, anomalie, isolamento	07 · 7–49	§5.3
`@Service`, `@Transactional(readOnly / isolation)`	07 · 55–57	tutti i service, `inserisciACatalogo`
Façade, un service per aggregato	03 · 50–51; 07 · 53	`SerieService.aggiungiPersonaggio`
`@Autowired`	03 · 61, 80	tutti i service e i controller
`@Controller`, `@GetMapping`, `Model`	03 · 56–66	tutti i controller MVC
Form, `th:object`, `th:field`	03-2 · 6–9	`formX.html`, `register.html`

Concetto	Slide	Dove in FigureDB
Redirect (PRG)	03-2 · 13–16	`return "redirect:…"`
Validazione lato client HTML5	08 · 9	`required`, `min`, `max`, `type="email"`
Bean Validation, `@Valid`, `BindingResult`	08 · 16–24	entità + controller
Messaggi di errore personalizzati	08 · 26–27	`message.properties`
Regole di business nel service	08 · 40–46, 56	`OperazioneNonValidaException`, `rejectValue`
`@ControllerAdvice` + `@ExceptionHandler`, 404/500	08 · 58–74	`GlobalExceptionHandler`, `templates/error/`
Logging SLF4J	08 · 76–87	`FigureService`, `GlobalExceptionHandler`
Fragments Thymeleaf	08 · 88–91	`fragments/navbar.html`
`StringTrimmerEditor` / `@InitBinder`	08 · 95–96	`GlobalBindingConfig`
`@RequestParam`, select multipla	08 · 97–98	`disegnatoriIds`, `aziendaId`
Paginazione	08 · 57, 75 (sommario)	`Page<Figure>`, `/figure?page=`
User + Credentials, ruoli	10 · 9–12	`model.User`, `model.Credentials`
UserDetailsService, BCrypt, `SecurityFilterChain`	10 · 13–19	`CustomUserDetailsService`, `SecurityConfiguration`
Utente loggato nei template	10 · 20–21	`GlobalController`, `sec:authorize`
CSRF	10 · 24–30	`th:action`, `csrf.ignoringRequestMatchers("/api/**")`
`@RestController`, JSON	11 · 21–22	`FigureRestController`
Cicli di serializzazione	11 · 23–28	`@JsonIgnore` + DTO
Metodi HTTP e codici di stato	11 · 12–15; 12-2 · 6	200 / 404 nelle API
TypeScript, interfacce, async/await	12-1 · 7–10	`types.ts`, `api.ts`
Axios	12-1 · 12–13	`api.ts`
Componenti, JSX, `key`	12-1 · 15–19	`FigureSearchWidget`
`useState`, `useEffect`, `useMemo`	12-1 · 24–32	`FigureSearchWidget`
Componenti controllati	12-2 · 13–14	input e select dei filtri
CORS	12-2 · 8	`corsConfigurationSource` (per Vite su 5173)
Sessione vs JWT	12-3 · 4–11	sessione + cookie `JSESSIONID`
Flexbox, variabili CSS, responsive	09	`stile.css`